

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HCM

KHOA ĐIỆN – ĐIỆN TỬ

BỘ MÔN KỸ THUẬT ĐIỆN TỬ-THÔNG TIN



HCMUTE

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG XE TỰ HÀNH BÁM SÁT MỤC
TIÊU SỬ DỤNG RASPBERRY PI**

GVHD: PhD. Trương Ngọc Sơn

Sinh viên:

TRẦN NHẬT NHÂN MSSV: 22139048

ĐINH NGỌC TRÂM ANH MSSV: 22139001

TP Hồ Chí Minh, tháng 7 năm 2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN ĐIỆN TỬ
BỘ MÔN KỸ THUẬT ĐIỆN TỬ-THÔNG TIN

ĐỒ ÁN TỐT NGHIỆP

**HỆ THỐNG XE TỰ HÀNH BẮM SÁT MỤC
TIÊU SỬ DỤNG RASPBERRY PI**

NGÀNH HỆ THỐNG NHÚNG VÀ IOT

Sinh viên: **TRẦN NHẤT NHÂN**

MSSV: 22139048

ĐINH NGỌC TRÂM ANH

MSSV: 22139001

TP. HỒ CHÍ MINH – 6/2026

THÔNG TIN ĐỒ ÁN TỐT NGHIỆP

1. THÔNG TIN SINH VIÊN

- Họ và tên sinh viên: Trần Nhật Nhân
- Mã số sinh viên: 22139048
- Họ và tên sinh viên (nếu có): Đinh Ngọc Trâm Anh
- Mã số sinh viên (nếu có): 22139061
- Lớp:
- Ngành: Hệ thống nhúng và IoT
- Khoa: Điện - Điện tử
- Email:
- Số điện thoại: 0808650969

2. THÔNG TIN ĐỀ TÀI

- Tên đề tài (Tiếng Việt): Hệ thống xe tự hành bám sát mục tiêu sử dụng Raspberry Pi
- Tên đề tài (Tiếng Anh): Autonomous target-tracking car using Raspberry Pi
- Loại hình:

- ☐ Nghiên cứu
- ☐ Thiết kế
- ☒ Chế tạo
- ☐ Mô phỏng
- ☐ Khác:

- Thời gian thực hiện:

- Ngày giao đề tài: 5.../...3.../2026
- Ngày hoàn thành: 22.../...6.../2026

3. GIẢNG VIÊN HƯỚNG DẪN

- Họ và tên: Trương Ngọc Sơn
- Học hàm/Học vị: Tiến sĩ (PhD)
- Đơn vị công tác:
- Email:

5. TÓM TẮT NỘI DUNG ĐỒ ÁN

...Hệ thống xe tự hành bám sát mục tiêu sử dụng Raspberry Pi 4 kết nối Webcam và điều khiển động cơ thông qua STM32F103

7. KẾT QUẢ ĐẠT ĐƯỢC

- ☒ Báo cáo đồ án tốt nghiệp.
- ☒ Mô hình / Thiết bị / Phần mềm hoàn chỉnh.
- ☐ Mã nguồn chương trình.
- ☐ Kết quả mô phỏng và thực nghiệm.
- ☐ Bài báo khoa học (nếu có).
- ☐ Sản phẩm khác:

1.1 8. KIỂM TRA ĐẠO VĂN

- Phần mềm kiểm tra: Turnitin.....
- Ngày kiểm tra: 21...../6...../2026
- Tỷ lệ tương đồng (Similarity Index): 2.5..... %
- Kết luận:
 - ☒ Đạt yêu cầu theo quy định của Nhà trường.
 - ☐ Không đạt yêu cầu theo quy định của Nhà trường.

9. CAM KẾT CỦA SINH VIÊN

Tôi cam kết rằng đồ án tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn. Các tài liệu tham khảo, số liệu, hình ảnh và kết quả nghiên cứu của người khác đều được trích dẫn đầy đủ theo đúng quy định. Tôi hoàn toàn chịu trách nhiệm trước Nhà trường về tính trung thực và tính hợp pháp của nội dung đồ án.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)


Trần Nhất Nhân

BẢN NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

Họ và tên sinh viên/MSSV: Tôn Nhất Nhân / 22139648

Họ và tên sinh viên/MSSV: Đinh Ngọc Trâm Anh / 22139001

Ngành: Hệ thống nhúng cơ TĐT

Tên đề tài: Hệ thống xử lý hành bám vật mục tiêu sử dụng Raspberry Pi

Giảng viên hướng dẫn: Trương Ngọc Sơn

1. Nhận xét chung về tinh thần, thái độ làm việc, nội dung và kết quả thực hiện đề tài

.....
.....

3. Đánh giá chung: Mức độ hoàn thành các mục tiêu đề ra:

☐ Xuất sắc ☐ Tốt ☐ Khá ☐ Đạt yêu cầu ☐ Chưa đạt yêu cầu

4. Kết luận

☒ Đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.

☐ Không đồng ý cho sinh viên bảo vệ Đồ án Tốt nghiệp trước Hội đồng đánh giá.

Lý do (nếu không đồng ý):

.....

TP. Hồ Chí Minh, ngày 22 tháng 06 năm 2026

GIẢNG VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)



Trương Ngọc Sơn

LỜI CẢM ƠN

Trong suốt thời gian học tập, nghiên cứu và thực hiện đồ án tốt nghiệp với đề tài "**Hệ thống xe tự hành bám sát mục tiêu sử dụng Raspberry Pi**", chúng em đã nhận được rất nhiều sự quan tâm, giúp đỡ và tạo điều kiện từ nhà trường, quý thầy cô, gia đình và bạn bè.

Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất đến ThS. Trương Ngọc Sơn. Thầy là người đã trực tiếp hướng dẫn, định hướng nghiên cứu và tận tình chỉ bảo chúng em trong suốt thời gian qua. Những kiến thức chuyên môn sâu rộng, sự kiên nhẫn giải đáp các thắc mắc cùng những lời động viên quý báu của thầy chính là ngọn đèn soi sáng, giúp chúng em định hình được phương pháp giải quyết vấn đề và vượt qua những khó khăn trong quá trình thiết kế, thi công cũng như lập trình hệ thống.

Chúng em cũng xin gửi lời cảm ơn chân thành đến Ban Giám hiệu nhà trường, cùng toàn thể quý thầy cô Khoa Điện Điện tử, chuyên ngành Hệ thống Nhúng và IoT, đã tận tâm giảng dạy, truyền đạt những nền tảng kiến thức vững chắc cho chúng em trong suốt những năm tháng ngồi trên giảng đường.

Mặc dù đã nỗ lực hết mình, nhưng do những hạn chế về mặt thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn vẫn còn những thiếu sót nhất định. Chúng em rất mong nhận được sự thông cảm, cũng như những lời nhận xét, góp ý từ quý thầy cô trong Hội đồng bảo vệ để đề tài được hoàn thiện hơn, đồng thời giúp chúng em đúc kết được những bài học kinh nghiệm quý giá cho chặng đường công việc sau này.

Kính chúc ThS. Trương Ngọc Sơn cùng toàn thể quý thầy cô luôn dồi dào sức khỏe, hạnh phúc và gặt hái được nhiều thành công hơn nữa trong sự nghiệp trồng người cao quý.

Chúng em xin chân thành cảm ơn!

MỤC LỤC

DANH MỤC HÌNH.....	8
DANH MỤC BẢNG.....	9
CÁC TỪ VIẾT TẮT	10
1.1 GIỚI THIỆU	11
1.2 MỤC TIÊU ĐỀ TÀI.....	12
1.2.1 Mục tiêu tổng quát	12
1.2.2 Mục tiêu cụ thể.....	12
1.3 GIỚI HẠN ĐỀ TÀI.....	13
1.4 PHƯƠNG PHÁP NGHIÊN CỨU.....	14
1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU	15
1.5.1 Đối tượng nghiên cứu.....	15
1.5.2 Phạm vi nghiên cứu.....	15
1.6 BỐ CỤC QUYỀN BẢO CÁO	15
Chương 2 CƠ SỞ LÝ THUYẾT	17
2.1 TỔNG QUAN VỀ HỆ THỐNG XE TỰ HÀNH BÁM THEO NGƯỜI.....	17
2.2 GIỚI THIỆU VỀ RASPBERRY PI 4 MODEL B.....	19
2.3. CƠ SỞ LÝ THUYẾT VỀ OBJECT DETECTION	20
2.3.1 Khái niệm phát hiện đối tượng.....	22
2.3.2 Bounding box và ý nghĩa trong đề tài	23
2.4 MÔ HÌNH PHÁT HIỆN NGƯỜI YOLO26N	25
2.4.1 Tổng quan YOLO	26
2.4.2 Kiến trúc YOLO26 và biến thể YOLO26n.....	27
2.4.3 Vai trò của YOLO26n trong hệ thống	29
2.5. CƠ SỞ LÝ THUYẾT VỀ PERSON RE-IDENTIFICATION	31
2.6. MÔ HÌNH OSNET_X0_25.....	33
2.7. TẬP DỮ LIỆU MARKET-1501	34
2.7.1 Tổng quan tập dữ liệu	34
2.7.2 Cấu trúc thư mục và các tập train/query/gallery	36
2.7.3 Ý nghĩa của person ID và camera ID	37
2.7.4 Vai trò của Market-1501 trong đề tài.....	40
Chương 3 THIẾT KẾ HỆ THỐNG.....	43
3.1 TỔNG QUAN THIẾT KẾ HỆ THỐNG.....	43
3.1.1 Mục tiêu thiết kế.....	43
3.1.2 Yêu cầu chức năng.....	44
3.2 KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG.....	46
3.2.1 Sơ đồ khối tổng thể	46
3.2.2 Chức năng các khối trong hệ thống.....	47
3.3 THIẾT KẾ PHẦN CỨNG	48
3.3.1 Chức năng của phần cứng.....	48
3.3.2 Sơ đồ khối phần cứng.....	50
3.3.3 Thiết kế từng khối	50

3.3.4 Tính toán thông số linh kiện và tổng công suất	51
3.4 THIẾT KẾ PHẦN MỀM	52
3.4.1 Kiến trúc phần mềm	52
3.4.2 Thiết kế ứng dụng điều khiển Flutter	56
3.5 THIẾT KẾ THUẬT TOÁN AI TRACKING	56
Chương 4 KẾT QUẢ	58
4.1 KẾT QUẢ MÔ HÌNH THI CÔNG.....	58
4.2 HOẠT ĐỘNG CỦA HỆ THỐNG	59
4.2.1 Hoạt động hiển thị luồng camera	60
4.2.2 Hoạt động cài đặt kết nối và thông số AI.....	61
4.2.3 Hoạt động ở chế độ thủ công	63
4.2.4 Hoạt động ở chế độ tự động	64
Chương 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	67
5.1 KẾT LUẬN.....	67
5.1.1 Kết quả đạt được so với mục tiêu đề tài.....	67
5.1.2 Ưu điểm của hệ thống	68
5.1.3 Hạn chế của hệ thống	69
5.1.4 Kết luận chung	70
5.2 HƯỚNG PHÁT TRIỂN.....	71
5.2.1 Tối ưu hiệu năng xử lý AI.....	71
5.2.2 Cải thiện thuật toán bám theo mục tiêu.....	73
5.2.3 Bổ sung chức năng tránh vật cản	74
5.2.4 Nâng cấp điều khiển động cơ và hệ thống nguồn	76
5.2.5 Hoàn thiện ứng dụng điều khiển và khả năng ứng dụng thực tế	77
TÀI LIỆU THAM KHẢO	79

DANH MỤC HÌNH

<i>Hình 2.1 Ảnh minh họa Raspberry Pi 4 Model B.....</i>	<i>20</i>
<i>Hình 2.2 Một số hình ảnh trong tập dữ liệu</i>	<i>35</i>
<i>Hình 3.1 Sơ đồ khối tổng thể của hệ thống</i>	<i>47</i>
<i>Hình 3.2 Sơ đồ khối cho hệ thống</i>	<i>50</i>
<i>Hình 3.3 Sơ đồ kiến trúc phần mềm tổng quát</i>	<i>53</i>
<i>Hình 3.4 Lưu đồ hoạt động tổng quát của phần mềm</i>	<i>55</i>
<i>Hình 4.1 Mô hình xe tự hành sau khi thi công hoàn thiện</i>	<i>58</i>
<i>Hình 4.2 Giao diện chờ kết nối camera từ Raspberry Pi</i>	<i>61</i>
<i>Hình 4.3 Giao diện cài đặt kết nối và thông số AI</i>	<i>62</i>
<i>Hình 4.4 Giao diện điều khiển thủ công trên điện thoại</i>	<i>63</i>
<i>Hình 4.5 Giao diện điều khiển thủ công trên máy tính</i>	<i>64</i>
<i>Hình 4.6 Giao diện chế độ tự động trên điện thoại.....</i>	<i>65</i>
<i>Hình 4.7 Kết quả phát hiện và theo dõi mục tiêu ở chế độ tự động</i>	<i>66</i>

DANH MỤC BẢNG

Bảng 2.1 Công nghệ và kiến trúc liên quan trực tiếp đến model/data.....	18
Bảng 2.2 Bảng phân bố các lớp chức năng trong hệ thống	19
Bảng 3.1 Chức năng các khối chính trong hệ thống.....	47
Bảng 3.2 Liệt kê thông số tiêu thụ điện năng của các linh kiện chính	51
Bảng 4.1 Kết quả thi công các khối chức năng của mô hình	59

CÁC TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ (Tiếng Anh)	Nghĩa tiếng Việt trong tài liệu
AI	Artificial Intelligence	Trí tuệ nhân tạo
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
DC	Direct Current	Động cơ điện một chiều (dùng cho động cơ)
DPM	Deformable Part Model	Thuật toán trích xuất hộp giới hạn tự động
FPS	Frames Per Second	Tốc độ khung hình
mAP	mean Average Precision	Độ chính xác trung bình
OSNet	Omni-Scale Network	Kiến trúc mạng nơ-ron học đặc trưng đa thang đo
PWM	Pulse Width Modulation	Bấm xung
Re-ID	Person Re-identification	Tái nhận dạng người
SLAM	Simultaneous Localization and Mapping	Lập bản đồ đồng thời
YOLO	You Only Look Once	Họ mô hình phát hiện đối tượng

Chương 1

GIỚI THIỆU

1.1 GIỚI THIỆU

Trong những năm gần đây, cùng với sự phát triển mạnh mẽ của cuộc Cách mạng công nghiệp lần thứ tư, các hệ thống robot thông minh và phương tiện tự hành đang ngày càng được quan tâm và ứng dụng rộng rãi trong nhiều lĩnh vực như logistics, thương mại bán lẻ, y tế, sản xuất công nghiệp và dịch vụ hỗ trợ con người ^[1]. Trong số đó, bài toán xây dựng xe tự hành có khả năng nhận diện và bám theo người dùng là một hướng nghiên cứu có tính ứng dụng cao, đặc biệt trong các kịch bản như xe chở hàng trong siêu thị, xe hỗ trợ vận chuyển trong kho hàng, robot hỗ trợ cá nhân hoặc các nền tảng robot dịch vụ trong môi trường trong nhà.

Về bản chất, xe tự hành bám theo người là một hệ thống tích hợp đa lĩnh vực, bao gồm thị giác máy tính, trí tuệ nhân tạo, điều khiển tự động, hệ thống nhúng và cơ điện tử. Để hệ thống có thể hoạt động hiệu quả, xe cần thực hiện đồng thời nhiều nhiệm vụ như thu nhận hình ảnh từ môi trường, phát hiện sự xuất hiện của con người, xác định đúng đối tượng mục tiêu cần theo dõi, duy trì khả năng bám theo trong quá trình đối tượng di chuyển, đồng thời chuyển đổi thông tin nhận được thành các lệnh điều khiển chuyển động phù hợp cho hệ thống truyền động của xe ^[2]. Đây là một bài toán tổng hợp, đòi hỏi sự phối hợp chặt chẽ giữa tầng cảm nhận, tầng xử lý thông minh và tầng điều khiển chấp hành.

Trong các hệ thống như vậy, việc lựa chọn nền tảng phần cứng phù hợp có ý nghĩa quyết định đối với khả năng triển khai thực tế. So với các vi điều khiển thông thường, Raspberry Pi 4 Model B có ưu thế về năng lực xử lý, khả năng chạy hệ

điều hành, hỗ trợ thư viện xử lý ảnh và mô hình học sâu, do đó phù hợp để đóng vai trò là bộ xử lý trung tâm cho hệ thống. Bên cạnh đó, các phần tử phần cứng như module điều khiển động cơ L298N, mạch giảm áp XL4015 và động cơ DC giảm tốc GA25-370 tạo thành nền tảng cơ bản để hiện thực hóa chuyển động của xe và bảo đảm hệ thống hoạt động ổn định ở mức nguồn và công suất.

Xuất phát từ nhu cầu thực tiễn nêu trên, đề tài “Hệ thống xe tự hành bám sát mục tiêu sử dụng Raspberry Pi” được thực hiện nhằm nghiên cứu một mô hình xe thông minh có khả năng nhận diện người, theo dõi mục tiêu và di chuyển bám theo người sử dụng trong thời gian thực. Đề tài không chỉ có ý nghĩa ứng dụng trong các bài toán vận chuyển hỗ trợ quy mô nhỏ mà còn góp phần làm rõ khả năng triển khai các thuật toán thị giác máy tính trên nền tảng nhúng chi phí hợp lý, từ đó tạo cơ sở cho các nghiên cứu phát triển sâu hơn trong lĩnh vực robot thông minh.

1.2 MỤC TIÊU ĐỀ TÀI

1.2.1 Mục tiêu tổng quát

Thiết kế và thi công một hệ thống xe tự hành có khả năng nhận diện, theo dõi và bám theo người mục tiêu trong thời gian thực, sử dụng nền tảng Raspberry Pi 4 Model B làm bộ xử lý trung tâm, kết hợp với mô hình phát hiện người YOLO26 và mô hình tái nhận dạng OSNet, hướng tới ứng dụng hỗ trợ vận chuyển hàng hóa trong môi trường trong nhà.

1.2.2 Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát nêu trên, đề tài đặt ra các mục tiêu cụ thể như sau:

Xây dựng mô hình phần cứng của xe tự hành bao gồm khối xử lý trung tâm (Raspberry Pi 4 Model B), khối truyền động (động cơ DC GA25-370 và mạch L298N), khối cấp nguồn (mạch giảm áp XL4015) và khối thu nhận hình ảnh (camera).

Tích hợp camera với Raspberry Pi để thu nhận dữ liệu hình ảnh liên tục theo thời gian thực phục vụ cho các tác vụ xử lý tiếp theo.

Triển khai mô hình phát hiện người YOLO26n trên Raspberry Pi để xác định vị trí và bounding box của tất cả đối tượng người trong khung hình.

Xây dựng pipeline tái nhận dạng người sử dụng mô hình trích xuất đặc trưng OSNet_x0_25, kết hợp với đo độ tương đồng cosine, nhằm duy trì nhận dạng đúng người mục tiêu trong môi trường có nhiều người xuất hiện đồng thời.

Thiết kế thuật toán điều khiển chuyển động của xe dựa trên sai lệch vị trí ngang của mục tiêu so với tâm khung hình và kích thước bounding box tương đối, tạo ra các lệnh tiến, lùi, rẽ trái, rẽ phải và dừng tương ứng.

Đánh giá hiệu năng hệ thống thông qua các tiêu chí định lượng như tốc độ xử lý (FPS), độ ổn định bám theo, khả năng tái nhận dạng mục tiêu sau khi bị che khuất và mức độ đáp ứng điều khiển trong các kịch bản thực nghiệm.

Đề xuất hướng cải tiến và tối ưu hóa hệ thống nhằm nâng cao hiệu năng và mở rộng khả năng ứng dụng thực tế.

1.3 GIỚI HẠN ĐỀ TÀI

Đề đảm bảo tính khả thi trong việc triển khai một hệ thống thực thể (prototype) trong khung thời gian và nguồn kinh phí của một đề án tốt nghiệp, đề tài tập trung giải quyết các bài toán cốt lõi và xác lập các giới hạn cụ thể như sau:

Môi trường thực nghiệm: Hệ thống được thiết kế để vận hành tối ưu trong môi trường nội thất (indoor), nơi có bề mặt sàn phẳng và điều kiện ánh sáng tán xạ tương đối ổn định. Đề tài chưa đi sâu vào việc xử lý các biến số ngoại cảnh phức tạp như cường độ ánh sáng thay đổi đột ngột (ánh nắng trực tiếp), địa hình gồ ghề hoặc các điều kiện thời tiết khắc nghiệt.

Đối tượng và kịch bản theo dõi: Thuật toán tập trung vào việc duy trì nhận diện và bám sát duy nhất một người dùng mục tiêu trong tầm nhìn (Field of View) của camera. Hệ thống chưa xử lý triệt để các tình huống bám sát trong đám đông có ngoại hình tương đồng cao hoặc khi mục tiêu bị che khuất hoàn toàn (total occlusion) trong thời gian dài.

Tối ưu hóa mô hình AI trên thiết bị biên: Do đặc tính kỹ thuật của vi máy tính Raspberry Pi 4 Model B có giới hạn về năng lực tính toán so với các máy tính chuyên dụng, đề tài chấp nhận sự đánh đổi giữa độ chính xác và tốc độ suy luận (Inference Speed). Hệ thống ưu tiên sử dụng các cấu trúc mạng nơ-ron tinh gọn

(như YOLO26 phiên bản Nano và OSNet rút gọn) thay vì các mô hình có tham số quá lớn.

Cơ chế điều hướng và an toàn: Xe di chuyển dựa trên phản hồi thị giác (Visual Servoing) để điều chỉnh hướng và vận tốc bám theo người^[3]. Trong phạm vi nghiên cứu này, hệ thống chưa tích hợp các thuật toán điều hướng cao cấp như lập bản đồ đồng thời (SLAM) hoặc xây dựng quỹ đạo tránh vật cản động bằng các cảm biến chuyên dụng như LiDAR.

Cấu hình phần cứng: Hệ thống được xây dựng dựa trên các linh kiện điện tử phổ biến như mạch cầu H L298N, động cơ DC giảm tốc GA25-370 và mạch giảm áp XL4015. Việc lựa chọn này nhằm tối ưu hóa chi phí và tính khả thi trong thi công mô hình, do đó hiệu suất vận hành (như tải trọng lớn hoặc độ bền công nghiệp) chưa phải là mục tiêu ưu tiên hàng đầu của đồ án.

1.4 PHƯƠNG PHÁP NGHIÊN CỨU

Để hoàn thành các mục tiêu trên, đề tài áp dụng các phương pháp nghiên cứu sau:

Nghiên cứu lý thuyết: Tổng hợp các tài liệu học thuật về xử lý ảnh, cấu trúc mạng nơ-ron tích chập (CNN), đặc biệt là các kiến trúc nhẹ dành cho thiết bị di động. Nghiên cứu lý thuyết về giao tiếp không dây Wi-Fi, giao thức Socket.IO dùng để truyền lệnh điều khiển thời gian thực, giao thức HTTP MJPEG dùng để truyền luồng camera từ Raspberry Pi về ứng dụng Flutter, giao tiếp UART giữa Raspberry Pi và STM32, đồng thời nghiên cứu phương pháp điều khiển động cơ DC bằng PWM.

Phân tích và Thiết kế: Phân tích yêu cầu hệ thống để chia nhỏ thành các module độc lập (Motor, Camera, AI Models). Thiết kế sơ đồ khối, sơ đồ nguyên lý mạch điện và lưu đồ thuật toán cho từng chế độ hoạt động.

Thi công và Lập trình: Lắp ráp khung xe phần cứng. Lập trình bằng ngôn ngữ Python để nhúng các mô hình AI và thuật toán điều khiển lên Raspberry Pi 4.

Thực nghiệm và Đánh giá: Chạy thử nghiệm xe trên sa bàn thực tế. Đánh giá độ trễ của hệ thống, độ chính xác (mAP) của các mô hình nhận diện, từ đó tinh chỉnh siêu tham số để tối ưu hiệu suất.

1.5 ĐỐI TƯỢNG VÀ PHẠM VI NGHIÊN CỨU

1.5.1 Đối tượng nghiên cứu

Đề tài nghiên cứu các đối tượng chính sau: xe tự hành cỡ nhỏ dùng cấu trúc truyền động vi sai hai bánh; thuật toán phát hiện và theo dõi người trong video thời gian thực; mô hình tái nhận dạng người dựa trên đặc trưng học sâu và độ tương đồng cosine; nền tảng Raspberry Pi 4 Model B cùng các thư viện AI tương ứng và cơ chế điều khiển động cơ DC qua mạch L298N với tín hiệu PWM..

1.5.2 Phạm vi nghiên cứu

Hệ thống hoạt động trong không gian trong nhà, ánh sáng ổn định, nền sàn phẳng. Đối tượng mục tiêu là người đi bộ ở khoảng cách 0,5m đến 3m trước camera. Tốc độ di chuyển được giới hạn ở mức đi bộ bình thường; các trường hợp chạy nhanh hay di chuyển đột ngột chưa được xét đến. Đề tài không đề cập đến tự lập bản đồ (SLAM), tránh vật cản tự động hay điều kiện ngoài trời.

1.6 BỐ CỤC QUYỀN BẢO CÁO

Báo cáo đề tài được tổ chức thành năm chương với nội dung như sau.

Chương 1 - Tổng quan: Trình bày bối cảnh, lý do thực hiện đề tài, xác định mục tiêu tổng quát và mục tiêu cụ thể, phạm vi nghiên cứu, phương pháp thực hiện và ý nghĩa khoa học cũng như thực tiễn của đề tài.

Chương 2 - Cơ sở lý thuyết: Trình bày các kiến thức lý thuyết nền tảng phục vụ cho việc thiết kế hệ thống, bao gồm lý thuyết robot di động bánh xe, điều khiển động cơ DC, nền tảng Raspberry Pi, xử lý ảnh số, mô hình phát hiện đối tượng, thuật toán theo dõi và tái nhận dạng người, cùng các nguyên lý điều khiển chuyển động của xe.

Chương 3 - Thiết kế hệ thống: Trình bày chi tiết quá trình thiết kế và thi công hệ thống, gồm yêu cầu kỹ thuật, kiến trúc tổng thể, thiết kế phần cứng, thiết kế phần mềm và các thuật toán điều khiển được triển khai.

Chương 4 - Kết quả thực nghiệm và đánh giá: Trình bày kết quả thi công mô hình thực tế, kết quả thực nghiệm trong các kịch bản kiểm thử, đánh giá định lượng hiệu năng hệ thống và phân tích các ưu điểm, hạn chế còn tồn tại.

Chương 5 - Kết luận và hướng phát triển: Tổng kết các kết quả đã đạt được so với mục tiêu đề ra, đưa ra kết luận chung và đề xuất các hướng nghiên cứu và phát triển tiếp theo.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 TỔNG QUAN VỀ HỆ THỐNG XE TỰ HÀNH BẮM THEO NGƯỜI

Trong hệ thống bám theo người, camera cung cấp chuỗi ảnh liên tục theo thời gian. Mỗi khung hình có thể chứa một người, nhiều người hoặc không có người nào. Hệ thống cần xử lý từng khung hình để xác định vị trí của các đối tượng người và sau đó quyết định đâu là người mục tiêu cần bám theo. Vì vậy, bài toán không chỉ là phát hiện sự xuất hiện của người, mà còn là duy trì đúng định danh của một người cụ thể trong suốt quá trình video diễn ra.

Nếu chỉ dùng phát hiện đối tượng, hệ thống sẽ biết được trong ảnh có những người nào và vị trí bounding box của từng người. Tuy nhiên, phát hiện đối tượng không trả lời được câu hỏi người trong bounding box hiện tại có phải là người đã được chọn ở khung hình trước hay không. Khi có người lạ đi ngang, khi mục tiêu bị che khuất ngắn hạn, hoặc khi nhiều người có kích thước gần nhau trong khung hình, việc chỉ dựa vào bounding box lớn nhất hoặc gần tâm ảnh có thể khiến hệ thống bám nhầm.

Để giải quyết hạn chế đó, pipeline trong đề tài kết hợp object detection và person re-identification. Object detection đóng vai trò tạo danh sách ứng viên người trong mỗi khung hình. Person Re-ID đóng vai trò gán mức độ giống nhau giữa từng ứng viên hiện tại với đặc trưng của người mục tiêu đã khóa trước đó. Nhờ vậy, hệ thống có thể chọn một bounding box duy nhất tương ứng với mục tiêu, thay vì xử lý tất cả người phát hiện được như nhau.

Về mặt dữ liệu, đầu vào của pipeline là ảnh màu theo định dạng video frame. Trong mã nguồn, OpenCV đọc từng frame từ video, YOLO26n nhận frame đầy đủ

để dự đoán bounding box người, sau đó từng vùng người được cắt ra từ frame gốc. Các ảnh crop này được chuyển từ BGR sang RGB, biến đổi thành ảnh PIL, resize về kích thước 256 x 128, chuyển thành tensor và chuẩn hóa trước khi đưa vào OSNet.

Đầu ra trung gian của YOLO là tập bounding box có dạng $x1, y1, x2, y2$. Đầu ra của OSNet là vector đặc trưng biểu diễn ngoại hình người. Đầu ra cuối cùng của bước nhận dạng mục tiêu là bounding box có độ tương đồng cosine cao nhất so với vector đặc trưng mục tiêu và vượt qua ngưỡng chấp nhận. Trong code, ngưỡng tương đồng đang được đặt là 0.60. Nếu không có ứng viên nào vượt ngưỡng, hệ thống không vẽ khung mục tiêu cho frame đó, điều này giúp giảm nguy cơ xác nhận sai người.

Bảng 2.1 Công nghệ và kiến trúc liên quan trực tiếp đến model/data

Nhóm	Công nghệ/kiến trúc	Vai trò trong code và báo cáo
Mô hình detection	YOLO26n / Ultralytics	Phát hiện tất cả đối tượng người trong từng frame; dùng trọng số yolo26n.pt; lấy class person bằng classes=0.
Mô hình Re-ID	OSNet_x0_25	Trích xuất embedding ngoại hình cho từng crop người; là mô hình được huấn luyện trên Market-1501.
Dataset train	Market-1501	Benchmark Person Re-ID dùng để huấn luyện OSNet; Torchreid đọc theo cấu trúc market1501.
Framework training	PyTorch	Nền tảng tensor, GPU/CUDA, checkpoint và export ONNX.
Thư viện Re-ID	Torchreid / deep-person-reid	Cung cấp ImageDataManager, build_model, optimizer, scheduler và ImageSoftmaxEngine.

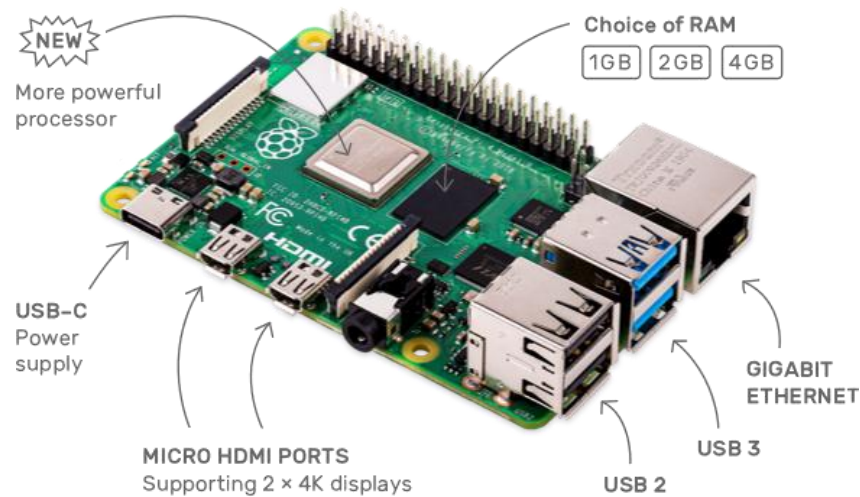
Nhóm	Công nghệ/kiến trúc	Vai trò trong code và báo cáo
Tiền xử lý ảnh	torchvision transforms + PIL	Resize crop người về 256 x 128, ToTensor và Normalize theo mean/std ImageNet.
Xử lý video	OpenCV	Đọc video, tách frame, cắt crop, chuyển BGR sang RGB, vẽ bbox và ghi video kết quả.
Tính toán vector	NumPy	Flatten embedding và tính cosine similarity.
Tối ưu triển khai	ONNX opset 18	Xuất OSNet đã train sang file .onnx để thuận tiện cho suy luận trên runtime khác.
Môi trường train	Kaggle + CUDA	Cung cấp đường dẫn dataset, GPU và nơi lưu checkpoint/log huấn luyện.

Bảng 2.2 Bảng phân bố các lớp chức năng trong hệ thống

Lớp chức năng	Đầu vào	Vai trò trong hệ thống
Cảm nhận	Luồng ảnh từ camera	Thu nhận thông tin môi trường theo thời gian thực để làm đầu vào cho toàn bộ chuỗi xử lý.
Nhận thức	Ảnh sau khi tiền xử lý	Phát hiện người, lựa chọn mục tiêu và cập nhật trạng thái mục tiêu qua từng khung hình.
Điều khiển	Tọa độ mục tiêu và các tham số hình học	Sinh lệnh điều khiển động cơ giúp xe giữ mục tiêu trong vùng quan sát mong muốn.

2.2 GIỚI THIỆU VỀ RASPBERRY PI 4 MODEL B

Trong các hệ thống robot tự hành hiện đại, việc xử lý lượng lớn dữ liệu hình ảnh và thực thi các mô hình trí tuệ nhân tạo đòi hỏi một trung tâm tính toán có hiệu năng vượt trội so với các dòng vi điều khiển truyền thống. Raspberry Pi 4 Model B được lựa chọn làm "bộ não" của hệ thống để đáp ứng những yêu cầu khắt khe này.



Hình 2.1 Ảnh minh họa Raspberry Pi 4 Model B

Raspberry Pi 4 là một máy tính kích thước nhỏ gọn nhưng sở hữu sức mạnh xử lý tiệm cận với máy tính cá nhân với giá thành không quá đắt đỏ. Chip được sử dụng trên Raspberry Pi 4 là Broadcom BCM2711, cùng với CPU ARM Cortex-A72x4 dựa trên kiến trúc 64-bit. Sự kết hợp giữa tốc độ xung nhịp cao và nền tảng RAM chuẩn LPDDR4 dao động từ 2GB đến 8GB cho phép thiết bị vận hành mượt mà hệ điều hành dựa trên nhân Linux. Vậy nên Raspberry Pi 4 có khả năng giúp hệ thống có chạy đa luồng các tác vụ nặng như thu thập video từ camera, xử lý ảnh bằng thư viện OpenCV và nội suy mạng nơ-ron với tốc độ khung hình ổn định.

Khác với các vi điều khiển cấp thấp, Raspberry Pi 4 được tích hợp sẵn các module truyền thông không dây mạnh mẽ ngay trên bo mạch. Thiết bị hỗ trợ kết nối mạng Wi-Fi băng tần kép và chuẩn Bluetooth 5.0. Raspberry Pi 4 Model B được tích hợp kết nối Wi-Fi băng tần kép và Bluetooth 5.0. Trong phạm vi đề tài, hệ thống sử dụng kết nối Wi-Fi làm kênh giao tiếp chính giữa Raspberry Pi và ứng dụng Flutter. Trên nền kết nối Wi-Fi, Raspberry Pi triển khai server để cung cấp luồng video theo dạng HTTP MJPEG cho ứng dụng hiển thị, đồng thời sử dụng Socket.IO để nhận lệnh điều khiển, cấu hình chế độ hoạt động và các tham số AI từ ứng dụng Flutter. Bluetooth là tính năng phần cứng có sẵn của Raspberry Pi nhưng không được sử dụng làm kênh điều khiển chính trong hệ thống này.

2.3. CƠ SỞ LÝ THUYẾT VỀ OBJECT DETECTION

Object Detection, hay phát hiện đối tượng, là một bài toán quan trọng trong lĩnh vực thị giác máy tính. Khác với bài toán phân loại ảnh, trong đó mô hình chỉ cần xác định ảnh thuộc lớp nào, Object Detection yêu cầu mô hình vừa xác định đối tượng xuất hiện trong ảnh là gì, vừa xác định đối tượng đó nằm ở vị trí nào trong khung hình. Do đó, đầu ra của một mô hình phát hiện đối tượng thường bao gồm nhãn lớp của đối tượng, vùng bao quanh đối tượng và độ tin cậy của kết quả dự đoán ^[4].

Trong đề tài hệ thống xe tự hành bám sát mục tiêu, Object Detection đóng vai trò là bước xử lý đầu tiên trong pipeline thị giác máy tính. Camera gắn trên xe liên tục thu nhận các khung hình từ môi trường phía trước. Mỗi khung hình này được đưa vào mô hình phát hiện đối tượng để tìm ra tất cả các đối tượng thuộc lớp người. Kết quả phát hiện người sau đó được sử dụng làm đầu vào cho các bước xử lý tiếp theo, bao gồm cắt vùng ảnh người, trích xuất đặc trưng bằng mô hình tái nhận dạng người và so khớp với người mục tiêu đã được hệ thống khóa trước đó.

Trong hệ thống được xây dựng, mô hình phát hiện đối tượng không chịu trách nhiệm xác định người nào là người mục tiêu. Nhiệm vụ chính của nó là phát hiện tất cả các người xuất hiện trong khung hình và trả về vị trí tương ứng của từng người. Việc xác định đâu là người cần bám theo sẽ được thực hiện ở tầng xử lý sau bằng mô hình tái nhận dạng người OSNet_x0_25 và phép đo độ tương đồng cosine. Vì vậy, Object Detection trong đề tài có thể được xem là bước tạo danh sách ứng viên ban đầu cho quá trình nhận dạng mục tiêu.

Về mặt tổng quát, một hệ thống Object Detection thường tạo ra ba loại thông tin chính cho mỗi đối tượng được phát hiện: bounding box, class label và confidence score. Bounding box cho biết vùng không gian trong ảnh chứa đối tượng. Class label cho biết đối tượng đó thuộc lớp nào, ví dụ như người, xe, động vật hoặc vật thể khác. Confidence score thể hiện mức độ tin cậy của mô hình đối với kết quả dự đoán. Trong phạm vi đề tài, hệ thống chỉ quan tâm đến lớp người, do đó các lớp đối tượng khác không được sử dụng trong pipeline xử lý chính.

Đối với bài toán bám theo người mục tiêu, Object Detection có ảnh hưởng trực tiếp đến chất lượng hoạt động của toàn hệ thống. Nếu mô hình phát hiện người bỏ sót người mục tiêu, các bước phía sau sẽ không có vùng ảnh người để xử lý. Ngược lại, nếu bounding box bị lệch, cắt thiếu cơ thể hoặc chứa quá nhiều vùng nền, đặc trưng ngoại hình được trích xuất bởi mô hình Re-ID có thể bị sai lệch. Do đó, chất lượng phát hiện đối tượng ở tầng đầu tiên là yếu tố quan trọng quyết định độ ổn định của quá trình khóa và bám mục tiêu.

Trong notebook triển khai của đề tài, bước phát hiện người được thực hiện bằng mô hình YOLO26n. Mô hình này nhận đầu vào là từng frame video đầy đủ và trả về danh sách bounding box của các đối tượng người. Sau đó, từng bounding box sẽ được dùng để crop vùng ảnh người và đưa sang mô hình OSNet_x0_25 nhằm trích xuất embedding ngoại hình. Pipeline này phù hợp với định hướng của đề tài vì YOLO26n đóng vai trò tạo vùng ứng viên người, còn OSNet_x0_25 đóng vai trò phân biệt người mục tiêu với các người khác trong cùng khung hình

2.3.1 Khái niệm phát hiện đối tượng

Phát hiện đối tượng là bài toán xác định đồng thời vị trí và loại đối tượng trong một ảnh hoặc một khung hình video. Với một ảnh đầu vào, mô hình Object Detection sẽ phân tích nội dung ảnh và dự đoán các vùng có khả năng chứa đối tượng. Mỗi vùng phát hiện được thường được biểu diễn bằng một hình chữ nhật bao quanh đối tượng, gọi là bounding box. Bên cạnh đó, mô hình cũng gán cho mỗi bounding box một nhãn lớp và một điểm tin cậy.

Khác với Image Classification, Object Detection có mức độ phức tạp cao hơn vì trong một ảnh có thể xuất hiện nhiều đối tượng cùng lúc. Các đối tượng này có thể khác nhau về kích thước, vị trí, góc nhìn, mức độ che khuất và điều kiện ánh sáng. Trong bài toán bám theo người, một khung hình có thể chỉ có một người, nhiều người hoặc không có người nào. Vì vậy, hệ thống cần một mô hình có khả năng phát hiện nhiều đối tượng người trong cùng một frame thay vì chỉ phân loại toàn bộ frame thành một nhãn duy nhất.

Đầu ra thứ nhất của mô hình phát hiện đối tượng là **bounding box**. Bounding box là vùng hình chữ nhật bao quanh đối tượng được phát hiện trong ảnh. Vùng

này giúp hệ thống biết được đối tượng nằm ở đâu trong khung hình. Trong đề tài, bounding box của người có vai trò rất quan trọng vì nó được dùng để cắt vùng ảnh người ra khỏi frame gốc trước khi đưa vào mô hình tái nhận dạng người.

Đầu ra thứ hai là **class label**, tức nhãn lớp của đối tượng. Nhãn lớp cho biết đối tượng được mô hình nhận diện thuộc nhóm nào. Ví dụ, với các mô hình được huấn luyện trên bộ dữ liệu COCO, lớp person thường được sử dụng để biểu diễn đối tượng con người. Trong đề tài, hệ thống chỉ cần phát hiện người nên các lớp đối tượng khác như xe, ghế, balo hoặc động vật không được đưa vào bước xử lý tiếp theo.

Đầu ra thứ ba là **confidence score**, tức điểm tin cậy của dự đoán. Điểm này thể hiện mức độ chắc chắn của mô hình rằng một bounding box chứa đối tượng thuộc lớp đã dự đoán. Confidence score càng cao thì mô hình càng tin rằng kết quả phát hiện là đúng. Trong các hệ thống thực tế, confidence score thường được dùng để lọc bỏ các dự đoán có độ tin cậy thấp, từ đó giảm số lượng phát hiện sai.

Trong phạm vi đề tài, Object Detection được sử dụng như một bước tiền xử lý cho bài toán tái nhận dạng người. Mô hình phát hiện đối tượng không cần biết danh tính thật của người trong ảnh. Nó chỉ cần xác định vùng nào trong frame có chứa người. Sau đó, từng vùng người này sẽ được đưa qua mô hình Re-ID để so sánh với đặc trưng của người mục tiêu. Cách chia pipeline thành hai tầng như vậy giúp hệ thống xử lý tốt hơn trong trường hợp có nhiều người xuất hiện đồng thời.

2.3.2 Bounding box và ý nghĩa trong đề tài

Bounding box là một trong những thành phần quan trọng nhất của bài toán Object Detection. Đây là vùng hình chữ nhật bao quanh đối tượng trong ảnh, giúp biểu diễn vị trí của đối tượng bằng các tọa độ trong hệ trục ảnh. Thông thường, một bounding box được mô tả bằng bốn giá trị tọa độ: x_1, y_1, x_2, y_2 . Trong đó, x_1 và y_1 là tọa độ góc trên bên trái của hình chữ nhật, còn x_2 và y_2 là tọa độ góc dưới bên phải.

Từ bốn tọa độ này, hệ thống có thể tính được các thông tin hình học khác của đối tượng như chiều rộng, chiều cao, diện tích và tọa độ tâm của bounding box. Chiều rộng được tính bằng hiệu giữa x_2 và x_1 , còn chiều cao được tính bằng hiệu

giữa y_2 và y_1 . Tâm của bounding box có thể được xác định bằng trung bình cộng của hai tọa độ theo trục ngang và trục dọc. Những thông tin này không chỉ phục vụ cho việc crop ảnh mà còn có thể dùng để suy ra vị trí tương đối của người mục tiêu trong khung hình.

Trong đề tài, bounding box có vai trò đầu tiên là xác định vùng ảnh người để đưa vào mô hình OSNet_x0_25. Sau khi YOLO26n phát hiện các đối tượng người trong frame, hệ thống sử dụng tọa độ bounding box để cắt từng vùng người ra khỏi ảnh gốc. Vùng ảnh được cắt này được gọi là crop người. Crop người sau đó được tiền xử lý, resize về kích thước phù hợp và đưa vào mô hình OSNet để trích xuất vector đặc trưng ngoại hình.

Chất lượng của bounding box ảnh hưởng trực tiếp đến chất lượng của đặc trưng Re-ID. Nếu bounding box bao quanh người quá rộng, vùng crop sẽ chứa nhiều nền hoặc các vật thể không liên quan. Điều này có thể làm nhiễu đặc trưng ngoại hình được trích xuất. Ngược lại, nếu bounding box quá hẹp hoặc bị lệch, vùng crop có thể bị mất các chi tiết quan trọng như màu áo, màu quần, dáng người hoặc phụ kiện. Khi đó, vector đặc trưng của cùng một người có thể thay đổi nhiều giữa các frame, làm giảm độ ổn định của quá trình so khớp mục tiêu.

Bounding box còn có vai trò thứ hai là cung cấp thông tin vị trí của mục tiêu trong frame. Sau khi hệ thống xác định được bounding box nào tương ứng với người mục tiêu, vị trí của bounding box có thể được dùng để suy ra hướng di chuyển của xe. Ví dụ, nếu tâm bounding box lệch sang bên trái so với tâm khung hình, hệ thống có thể điều chỉnh xe rẽ trái để đưa mục tiêu về giữa khung hình. Nếu bounding box quá nhỏ, điều đó có thể cho thấy mục tiêu đang ở xa camera; nếu bounding box quá lớn, mục tiêu có thể đang ở quá gần. Vì vậy, bounding box không chỉ phục vụ cho nhận dạng mà còn là dữ liệu đầu vào quan trọng cho tăng điều khiển chuyển động.

Trong mã nguồn của đề tài, các bounding box quá nhỏ được loại bỏ trước khi đưa vào các bước xử lý tiếp theo. Cụ thể, nếu chiều rộng hoặc chiều cao của bounding box nhỏ hơn 15 pixel thì vùng phát hiện đó sẽ bị bỏ qua. Cách lọc này là cần thiết vì các vùng người quá nhỏ thường chứa rất ít thông tin ngoại hình, dễ

bị nhiễu bởi nền ảnh và có thể tạo ra embedding không ổn định khi đưa vào OSNet. Việc loại bỏ các bounding box quá nhỏ giúp giảm số lượng ứng viên không đáng tin cậy và làm cho quá trình so khớp mục tiêu ổn định hơn.

Trong pipeline suy luận của đề tài, YOLO26n phát hiện người trong từng frame và trả về danh sách bounding box. Các bounding box này sau đó được lọc, crop và đưa qua OSNet để tạo đặc trưng. Người có đặc trưng giống nhất với người mục tiêu sẽ được chọn dựa trên cosine similarity. Theo bảng pipeline đã tách riêng, sau khi chạy YOLO26n với classes=0, hệ thống bỏ các bounding box có chiều rộng hoặc chiều cao dưới 15 pixel, sau đó crop người, trích xuất đặc trưng và chọn ứng viên có độ tương đồng cao nhất.

2.4 MÔ HÌNH PHÁT HIỆN NGƯỜI YOLO26N

Trong hệ thống xe tự hành bám sát mục tiêu, mô hình phát hiện người đóng vai trò là tầng xử lý đầu tiên trong pipeline thị giác máy tính. Nhiệm vụ của tầng này là xác định trong mỗi khung hình video có những đối tượng người nào và vị trí của từng người trong ảnh. Kết quả phát hiện này được biểu diễn dưới dạng các bounding box bao quanh vùng chứa người. Các bounding box sau đó được sử dụng làm đầu vào cho các bước xử lý tiếp theo như cắt vùng ảnh người, trích xuất đặc trưng ngoại hình bằng mô hình OSNet_x0_25 và so khớp với người mục tiêu.

Trong đề tài, mô hình được sử dụng cho tác vụ phát hiện người là **YOLO26n**. Đây là biến thể nhỏ gọn của họ mô hình YOLO26, phù hợp với các hệ thống cần xử lý video theo thời gian thực và có giới hạn về tài nguyên tính toán. YOLO26n nhận đầu vào là từng frame video đầy đủ, sau đó trả về danh sách các đối tượng được phát hiện trong frame. Trong phạm vi đề tài, hệ thống chỉ quan tâm đến đối tượng thuộc lớp người, do đó các lớp đối tượng khác không được sử dụng trong pipeline chính.

Một điểm cần nhấn mạnh là YOLO26n trong hệ thống không thực hiện nhiệm vụ nhận diện chính xác người mục tiêu. Mô hình này chỉ xác định vị trí của tất cả người xuất hiện trong khung hình. Việc phân biệt đâu là người cần bám theo được thực hiện ở tầng sau bằng mô hình tái nhận dạng người OSNet_x0_25 và phép đo

độ tương đồng cosine. Do đó, YOLO26n có thể được xem là bước tạo danh sách ứng viên người để hệ thống tiếp tục xử lý.

Trong notebook triển khai, YOLO26n được nạp từ trọng số huấn luyện sẵn yolo26n.pt, thay vì được huấn luyện lại trên tập dữ liệu riêng. Điều này giúp rút ngắn thời gian phát triển hệ thống, đồng thời tận dụng khả năng phát hiện người đã được học từ các tập dữ liệu lớn. Phần huấn luyện chính của đề tài tập trung vào mô hình OSNet_x0_25 trên tập dữ liệu Market-1501, còn YOLO26n được dùng như một mô hình phát hiện người đã có sẵn để phục vụ pipeline suy luận

2.4.1 Tổng quan YOLO

YOLO, viết tắt của **You Only Look Once**, là một họ mô hình phát hiện đối tượng phổ biến trong lĩnh vực thị giác máy tính. Ý tưởng chính của YOLO là thực hiện phát hiện đối tượng trong một lần suy luận duy nhất trên toàn bộ ảnh đầu vào. Thay vì chia quá trình phát hiện thành nhiều giai đoạn phức tạp, YOLO trực tiếp dự đoán vị trí bounding box, nhãn lớp và độ tin cậy của đối tượng từ ảnh đầu vào^[5].

Các mô hình phát hiện đối tượng truyền thống thường có thể chia thành hai nhóm lớn: mô hình hai giai đoạn và mô hình một giai đoạn. Mô hình hai giai đoạn thường tạo ra các vùng đề xuất có khả năng chứa đối tượng trước, sau đó mới phân loại và hiệu chỉnh vị trí các vùng đó. Cách tiếp cận này có thể đạt độ chính xác cao nhưng thường có tốc độ xử lý chậm hơn. Ngược lại, YOLO thuộc nhóm mô hình một giai đoạn, trong đó mạng nơ-ron thực hiện dự đoán trực tiếp trên ảnh. Nhờ vậy, YOLO thường có tốc độ suy luận nhanh và phù hợp với các ứng dụng thời gian thực.

Đối với bài toán xe tự hành bám theo người, tốc độ xử lý là yếu tố rất quan trọng. Camera liên tục cung cấp các khung hình video, vì vậy hệ thống cần phát hiện người trong từng frame với độ trễ thấp. Nếu mô hình phát hiện quá chậm, xe sẽ phản ứng không kịp với sự thay đổi vị trí của người mục tiêu. Do đó, việc lựa chọn một mô hình phát hiện đối tượng có tốc độ cao như YOLO là phù hợp với yêu cầu của đề tài.

Đầu ra của YOLO bao gồm các bounding box bao quanh đối tượng, nhãn lớp tương ứng và điểm tin cậy. Với bài toán phát hiện người, bounding box cho biết

vùng ảnh chứa người trong frame, nhãn lớp cho biết đối tượng thuộc lớp person, còn điểm tin cậy thể hiện mức độ chắc chắn của mô hình đối với dự đoán đó. Từ các thông tin này, hệ thống có thể lọc ra các đối tượng người và bỏ qua các đối tượng không liên quan.

Trong hệ thống bám mục tiêu, YOLO không lưu trữ danh tính của từng người qua thời gian. Mô hình chỉ xử lý từng frame và trả về vị trí của các người xuất hiện trong frame đó. Điều này có nghĩa là nếu trong ảnh có nhiều người, YOLO có thể phát hiện tất cả họ nhưng không biết người nào là người đã được chọn ở frame trước. Vì vậy, YOLO cần được kết hợp với mô hình Re-ID để hệ thống có khả năng duy trì đúng người mục tiêu.

Tóm lại, YOLO là nền tảng quan trọng cho bước phát hiện người trong đề tài. Mô hình này giúp hệ thống nhanh chóng xác định các vùng ảnh có chứa người, từ đó tạo dữ liệu đầu vào cho quá trình tái nhận dạng người. Nhờ đặc điểm xử lý nhanh và trực tiếp trên toàn bộ frame, YOLO phù hợp với yêu cầu xử lý video thời gian thực của xe tự hành bám sát mục tiêu.

2.4.2 Kiến trúc YOLO26 và biến thể YOLO26n

YOLO26 là một phiên bản hiện đại trong họ mô hình YOLO, được phát triển với mục tiêu cải thiện khả năng phát hiện đối tượng trong nhiều điều kiện khác nhau. YOLO26 hỗ trợ nhiều tác vụ thị giác máy tính như object detection, instance segmentation, pose estimation và image classification. Trong phạm vi đề tài này, hệ thống chỉ sử dụng YOLO26 cho tác vụ **object detection**, cụ thể là phát hiện đối tượng người trong từng frame video.

Về mặt kiến trúc tổng quát, một mô hình YOLO26 có thể được hiểu gồm ba thành phần chính: backbone, neck và head. Backbone có nhiệm vụ trích xuất đặc trưng từ ảnh đầu vào. Các đặc trưng ban đầu thường là cạnh, màu sắc, kết cấu và hình dạng đơn giản; ở các tầng sâu hơn, mạng học được các đặc trưng phức tạp hơn như bộ phận cơ thể hoặc hình dáng tổng thể của người. Neck có nhiệm vụ kết hợp đặc trưng ở nhiều mức khác nhau để mô hình có thể phát hiện đối tượng với nhiều kích thước. Head là phần dự đoán cuối cùng, tạo ra bounding box, nhãn lớp và độ tin cậy cho từng đối tượng.

Trong bài toán phát hiện người, việc kết hợp đặc trưng nhiều mức là cần thiết vì kích thước người trong ảnh có thể thay đổi liên tục. Khi người đứng gần camera, bounding box của người có kích thước lớn. Khi người đi xa camera, vùng người trong ảnh trở nên nhỏ hơn. Ngoài ra, tư thế, góc nhìn và mức độ che khuất cũng có thể thay đổi qua từng frame. Vì vậy, kiến trúc phát hiện đối tượng cần có khả năng xử lý cả đối tượng lớn và nhỏ trong cùng một khung hình.

YOLO26 có nhiều biến thể với kích thước và độ phức tạp khác nhau, chẳng hạn như YOLO26n, YOLO26s, YOLO26m, YOLO26l và YOLO26x. Trong đó, ký hiệu n trong YOLO26n là viết tắt của **Nano**, chỉ phiên bản nhỏ nhất và nhẹ nhất. Các phiên bản lớn hơn thường có số tham số nhiều hơn và có thể đạt độ chính xác cao hơn trong một số trường hợp, nhưng đồng thời cũng yêu cầu nhiều tài nguyên tính toán hơn. Ngược lại, YOLO26n có ưu điểm là nhẹ, tốc độ suy luận nhanh và phù hợp với hệ thống có giới hạn phần cứng.

Trong đề tài, YOLO26n được lựa chọn vì hệ thống hướng đến xử lý video theo thời gian thực trên nền tảng nhúng. Khi mỗi frame được xử lý, hệ thống không chỉ chạy mô hình YOLO để phát hiện người mà còn phải crop từng vùng người và đưa qua mô hình OSNet để trích xuất đặc trưng. Nếu mô hình phát hiện đối tượng quá lớn, tổng thời gian xử lý của pipeline sẽ tăng lên, làm giảm FPS và khiến xe phản ứng chậm hơn với chuyển động của người mục tiêu. Vì vậy, YOLO26n là lựa chọn hợp lý để cân bằng giữa tốc độ và khả năng phát hiện.

Trong notebook triển khai, mô hình YOLO26n được khởi tạo bằng thư viện Ultralytics thông qua trọng số yolo26n.pt. Khi chạy suy luận, hệ thống truyền từng frame vào mô hình và chỉ lấy kết quả thuộc lớp person bằng tham số classes=0. Cách làm này giúp giảm số lượng đối tượng không cần thiết trong kết quả đầu ra, vì hệ thống không cần xử lý các lớp như xe, ghế, balo hoặc các vật thể khác. Nhờ đó, pipeline phía sau chỉ tập trung vào các vùng ảnh người.

Đầu ra của YOLO26n là danh sách các bounding box người trong frame. Mỗi bounding box được biểu diễn bởi bốn tọa độ x1, y1, x2, y2. Trong mã nguồn, các bounding box có chiều rộng hoặc chiều cao quá nhỏ sẽ bị loại bỏ, cụ thể là nhỏ hơn 15 pixel. Việc loại bỏ này giúp tránh đưa các vùng ảnh quá nhỏ hoặc không

đủ thông tin ngoại hình vào mô hình OSNet. Đây là bước quan trọng để làm giảm nhiễu trong quá trình tái nhận dạng người.

Như vậy, YOLO26n không chỉ được chọn vì khả năng phát hiện người mà còn vì tính phù hợp với kiến trúc tổng thể của hệ thống. Mô hình này đủ nhẹ để chạy trong pipeline video, đồng thời vẫn cung cấp được bounding box cần thiết cho bước Re-ID. Trong hệ thống bám sát mục tiêu, YOLO26n giữ vai trò là mô hình phát hiện người ở tầng đầu, giúp hệ thống xác định các vùng ảnh cần được xử lý tiếp theo.

YOLO26n không chỉ phát hiện người trong frame, mà còn cung cấp tọa độ, kích thước và vị trí tương đối của người đó. Các thông tin này được sử dụng để crop ảnh người đưa vào OSNet_x0_25, loại bỏ các vùng phát hiện quá nhỏ và hỗ trợ tầng điều khiển xác định hướng di chuyển của xe. Vì vậy, bounding box là cầu nối quan trọng giữa mô hình phát hiện người và các module tái nhận dạng, so khớp mục tiêu và điều khiển chuyển động trong hệ thống.

2.4.3 Vai trò của YOLO26n trong hệ thống

Trong hệ thống xe tự hành bám sát mục tiêu, YOLO26n có vai trò tạo ra đầu vào ban đầu cho toàn bộ quá trình nhận diện và bám theo người. Camera gắn trên xe thu nhận các khung hình liên tục từ môi trường. Mỗi frame được đưa vào YOLO26n để phát hiện tất cả đối tượng người xuất hiện trong ảnh. Kết quả của bước này là danh sách bounding box tương ứng với từng người.

Vai trò đầu tiên của YOLO26n là xác định vị trí của người trong frame. Nếu không có bước phát hiện người, hệ thống sẽ không biết vùng ảnh nào cần được đưa vào mô hình Re-ID. Việc xử lý toàn bộ frame bằng OSNet là không phù hợp, vì OSNet được thiết kế để nhận đầu vào là ảnh crop của từng người, không phải toàn bộ khung cảnh. Do đó, YOLO26n đóng vai trò như một bộ lọc không gian, giúp tách các vùng ảnh người ra khỏi ảnh nền.

Vai trò thứ hai của YOLO26n là tạo danh sách ứng viên người cho quá trình khóa mục tiêu. Trong một frame có thể có nhiều người xuất hiện cùng lúc. YOLO26n phát hiện tất cả các người này và trả về các bounding box tương ứng. Sau đó, từng bounding box được crop thành ảnh người riêng biệt và đưa vào

OSNet_x0_25 để trích xuất embedding. Các embedding này được so sánh với embedding của người mục tiêu bằng cosine similarity. Người có độ tương đồng cao nhất và vượt ngưỡng được chọn làm mục tiêu cần bám theo.

Vai trò thứ ba của YOLO26n là cung cấp thông tin vị trí phục vụ điều khiển chuyển động. Sau khi hệ thống xác định được bounding box nào thuộc về người mục tiêu, tọa độ của bounding box có thể được dùng để tính vị trí tương đối của người trong khung hình. Nếu tâm bounding box lệch sang trái hoặc sang phải so với tâm frame, hệ thống có thể điều chỉnh hướng di chuyển của xe. Nếu kích thước bounding box thay đổi, hệ thống có thể ước lượng tương đối khoảng cách giữa xe và người mục tiêu. Vì vậy, kết quả phát hiện của YOLO26n không chỉ phục vụ nhận dạng mà còn hỗ trợ tăng điều khiển.

Trong giai đoạn khóa mục tiêu ban đầu, hệ thống có thể chọn bounding box có diện tích lớn nhất trong frame đầu tiên có người làm người mục tiêu ban đầu. Giả định của phương pháp này là khi bắt đầu hoạt động, người dùng đứng gần camera và xuất hiện nổi bật nhất trong khung hình. Crop của bounding box này được đưa vào OSNet để tạo target_feature. Từ các frame sau, hệ thống không còn chọn mục tiêu chỉ dựa vào kích thước bounding box mà dùng cosine similarity để so khớp đặc trưng ngoại hình.

YOLO26n cũng góp phần giảm tải tính toán cho hệ thống. Thay vì đưa toàn bộ frame hoặc nhiều vùng ảnh không liên quan vào OSNet, hệ thống chỉ xử lý các vùng được YOLO xác định là người. Điều này giúp giảm số lượng ảnh crop cần trích xuất đặc trưng, từ đó cải thiện tốc độ của toàn pipeline. Đây là yếu tố quan trọng trong hệ thống thời gian thực, vì xe cần phản hồi nhanh khi người mục tiêu di chuyển.

Tuy nhiên, cần nhấn mạnh rằng YOLO26n không có khả năng phân biệt danh tính giữa các người. Nếu hai người cùng xuất hiện trong khung hình, YOLO26n chỉ gán cả hai là lớp person. Mô hình không biết người nào là người mục tiêu đã được khóa trước đó. Vì vậy, nếu chỉ sử dụng YOLO26n, hệ thống có thể bám nhầm người khi có người khác đi ngang qua, khi người mục tiêu bị che khuất hoặc khi

nhiều người đứng gần nhau. Đây là lý do hệ thống cần kết hợp thêm OSNet_x0_25 để tái nhận dạng người dựa trên đặc trưng ngoại hình.

Chất lượng hoạt động của YOLO26n ảnh hưởng trực tiếp đến các bước phía sau. Nếu YOLO26n bỏ sót người mục tiêu, OSNet sẽ không có crop đầu vào để so khớp. Nếu bounding box bị lệch hoặc cắt thiếu nhiều phần cơ thể, embedding được trích xuất có thể không ổn định, làm giảm độ chính xác của cosine similarity. Ngược lại, nếu YOLO26n phát hiện người chính xác và bounding box bao quanh người hợp lý, bước Re-ID phía sau sẽ có dữ liệu đầu vào tốt hơn, giúp hệ thống duy trì đúng mục tiêu ổn định hơn.

2.5. CƠ SỞ LÝ THUYẾT VỀ PERSON RE-IDENTIFICATION

Person Re-identification, thường được viết tắt là Person Re-ID, là bài toán tái nhận dạng một người đã xuất hiện trước đó dựa trên đặc trưng ngoại hình. Khác với bài toán phát hiện người, vốn chỉ xác định trong ảnh có người hay không và người đó nằm ở vị trí nào, Person Re-ID tập trung vào việc xác định xem hai ảnh người khác nhau có thuộc cùng một cá nhân hay không. Đây là một bài toán quan trọng trong các hệ thống giám sát, theo dõi mục tiêu và robot bám theo người [9].

Trong hệ thống xe tự hành bám sát mục tiêu, Person Re-ID được sử dụng để duy trì đúng danh tính của người cần bám theo trong suốt quá trình hoạt động. Khi camera thu nhận từng frame video, mô hình YOLO26n phát hiện tất cả đối tượng người trong khung hình và tạo ra các bounding box tương ứng. Tuy nhiên, YOLO26n chỉ cho biết vị trí của các người trong ảnh, không thể xác định người nào là người mục tiêu. Vì vậy, từng vùng ảnh người sau khi được crop từ bounding box sẽ được đưa vào mô hình Re-ID để trích xuất đặc trưng ngoại hình và so sánh với người mục tiêu đã được khóa trước đó.

Trong đề tài, mô hình Re-ID được sử dụng là **OSNet_x0_25**. Mô hình này nhận đầu vào là ảnh crop của từng người và tạo ra một vector đặc trưng, còn gọi là **embedding vector**. Vector này biểu diễn các đặc điểm ngoại hình của người trong ảnh, chẳng hạn như màu áo, màu quần, dáng người, phụ kiện hoặc các đặc điểm cục bộ khác. Sau đó, hệ thống so sánh vector của từng người trong frame

hiện tại với vector của người mục tiêu ^[10]. Người có độ tương đồng cao nhất và vượt qua ngưỡng chấp nhận sẽ được xem là mục tiêu cần bám theo.

Theo pipeline của đề tài, YOLO26n tạo bounding box người, còn OSNet_x0_25 trích xuất embedding ngoại hình để phục vụ quá trình so khớp mục tiêu bằng cosine similarity .

Về mặt toán học, một ảnh người sau khi đi qua mô hình Re-ID có thể được biểu diễn thành một vector đặc trưng như sau:

$$f = F(I) \quad (1)$$

Trong đó:

I : ảnh crop của người

F : mô hình trích xuất đặc trưng Re-ID, trong đề tài là OSNet_x0_25

f : embedding vector biểu diễn đặc trưng ngoại hình của ảnh người

Nếu ảnh người mục tiêu ban đầu được biểu diễn bởi vector: f_{target} , và một người ứng viên trong frame hiện tại được biểu diễn bởi vector: f_i , thì bài toán Re-ID trong hệ thống có thể được hiểu là tìm người ứng viên có đặc trưng gần nhất với người mục tiêu.

Một cách đo mức độ khác nhau giữa hai vector là khoảng cách Euclidean:

$$d(f_{target}, f_i) = ||f_{target} - f_i||_2 \quad (2)$$

Nếu giá trị d càng nhỏ thì hai vector càng gần nhau, nghĩa là hai ảnh có khả năng thuộc cùng một người cao hơn. Tuy nhiên, trong code của đề tài, hệ thống sử dụng **cosine similarity** thay vì khoảng cách Euclidean. Cosine similarity đo mức độ cùng hướng giữa hai vector đặc trưng và được tính theo công thức:

$$\text{sim}(f_{target}, f_i) = \frac{f_{target} \cdot f_i}{||f_{target}|| \times ||f_i||} \quad (3)$$

Trong đó:

$f_{target} \cdot f_i$: tích vô hướng giữa hai vector

$||f_{target}||$: độ lớn của vector mục tiêu

$||f_i||$: độ lớn của vector ứng viên

Giá trị cosine similarity càng gần 1 thì hai vector càng giống nhau. Nếu giá trị này thấp, hệ thống xem người ứng viên hiện tại không đủ giống với người mục tiêu. Trong notebook triển khai, hệ thống chọn người có độ tương đồng cosine cao nhất và lớn hơn ngưỡng 0.60 để làm mục tiêu trong frame hiện tại. Cách làm này giúp hạn chế trường hợp bám nhầm người khi có nhiều người cùng xuất hiện trong khung hình .

Như vậy, Person Re-ID là thành phần giúp hệ thống có khả năng duy trì đúng người mục tiêu, thay vì chỉ phát hiện tất cả người trong ảnh. Trong hệ thống xe tự hành bám sát mục tiêu, Re-ID đóng vai trò trung gian giữa tầng phát hiện người và tầng điều khiển chuyển động. YOLO26n giúp tìm vùng ảnh người, OSNet_x0_25 giúp trích xuất đặc trưng ngoại hình, còn cosine similarity giúp chọn đúng người mục tiêu trong số các ứng viên được phát hiện.

2.6. MÔ HÌNH OSNET_X0_25

Để duy trì đúng mục tiêu khi có nhiều người xuất hiện hoặc mục tiêu bị che khuất tạm thời, hệ thống sử dụng **OSNet_x0_25** để tái nhận dạng (Re-ID). Thay vì chỉ dựa vào vị trí như thuật toán theo dõi truyền thống, OSNet trích xuất "vân tay ngoại hình" (embedding vector) của người dùng^[11].

Trong mã nguồn, sau khi YOLO26n cung cấp các vùng ảnh người (crop), hệ thống đưa các ảnh này qua OSNet. Kết quả là một vector đặc trưng biểu diễn màu sắc trang phục, dáng người và các chi tiết hình thái. Hệ thống sẽ so khớp vector này với vector của mục tiêu ban đầu để đưa ra quyết định bám đuổi chính xác.

Hệ thống sử dụng độ tương đồng Cosine để so sánh giữa vector đặc trưng của mục tiêu (f_{target}) và vector của các ứng viên hiện tại (f_i):

Giá trị: Dao động từ -1 đến 1; giá trị càng gần 1, hai ngoại hình càng giống nhau.

Cơ chế khóa mục tiêu: Trong code, hệ thống chỉ xác nhận người đó là mục tiêu nếu $\text{sim} > 0.60$. Ngưỡng này giúp hệ thống tự động ngắt lệnh bám theo khi người mục tiêu khuất hẳn hoặc bị nhầm lẫn bởi người khác, giúp robot vận hành an toàn hơn.

2.7. TẬP DỮ LIỆU MARKET-1501

Để huấn luyện mô hình tái nhận dạng người, hệ thống cần một tập dữ liệu có ảnh của nhiều người khác nhau, được gắn nhãn định danh rõ ràng. Trong đề tài, tập dữ liệu được sử dụng cho quá trình huấn luyện OSNet_x0_25 là **Market-1501**. Đây là một trong những tập dữ liệu phổ biến cho bài toán Person Re-identification, thường được dùng để huấn luyện và đánh giá khả năng nhận dạng lại cùng một người trong các điều kiện camera khác nhau^[8].

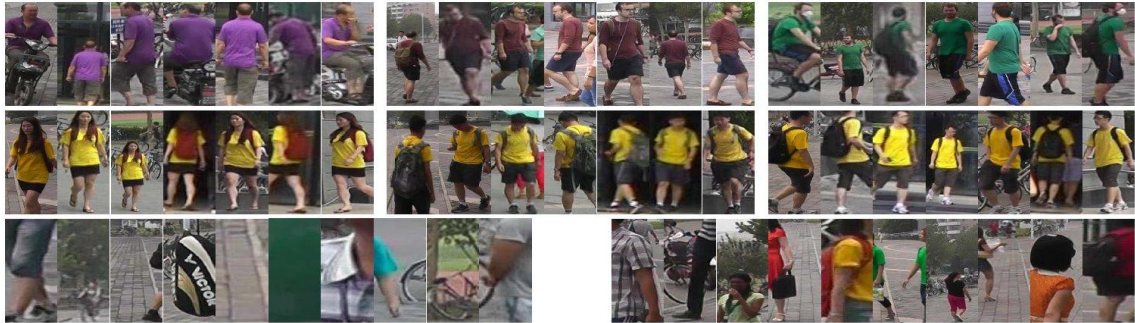
Market-1501 không phải là tập dữ liệu dùng để huấn luyện YOLO26n. Trong pipeline của đề tài, YOLO26n được sử dụng từ trọng số pretrained yolo26n.pt để phát hiện người. Phần huấn luyện chính nằm ở mô hình OSNet_x0_25, trong đó Market-1501 cung cấp các ảnh người đã được gắn nhãn person ID và camera ID. Vì vậy, khi trình bày Chương 2, cần nhấn mạnh rằng Market-1501 là tập dữ liệu phục vụ cho bài toán Re-ID, không phải tập dữ liệu huấn luyện mô hình phát hiện đối tượng.

Trong notebook triển khai, dữ liệu Market-1501 được đặt tại đường dẫn đầu vào của môi trường Kaggle, sau đó được liên kết đến cấu trúc thư mục mà Torchreid yêu cầu. Torchreid đọc dataset theo tên market1501, sau đó tự động quản lý các tập train, query và gallery phục vụ quá trình huấn luyện và đánh giá. Theo phân phụ lục cấu hình, Market-1501 được sử dụng làm source và target trong ImageDataManager, đồng thời là tập dữ liệu train chính cho OSNet_x0_25.

2.7.1 Tổng quan tập dữ liệu

Hiện nay, Market-1501 là một trong những tập dữ liệu ReID kinh điển và được trích dẫn rộng rãi nhất. Cụ thể, tập dữ liệu cung cấp 32.668 hình ảnh của 1.501 cá nhân khác nhau. Các hình ảnh này được thu thập từ mạng lưới 6 camera quan sát

(bao gồm 5 camera độ phân giải cao và 1 camera độ phân giải thấp) với các góc quay chéo nhau tại khuôn viên trường đại học. Một điểm nhấn học thuật quan trọng của Market-1501 là các hộp giới hạn chứa người không được cắt xén thủ công hoàn hảo, mà được trích xuất tự động bằng thuật toán DPM (Deformable Part Model). Việc bao gồm cả các lỗi nhận diện từ DPM giúp mô phỏng chính xác nhiều từ môi trường thực tế, từ đó tăng cường tính mạnh mẽ cho hệ thống. Bên cạnh đó, tập dữ liệu còn chứa hơn 500.000 ảnh nhiễu để thử thách để thử thách và đánh giá các độ đo chuẩn xác như mAP (mean Average Precision) và Rank-1 accuracy.



Hình 2.2 Một số hình ảnh trong tập dữ liệu

Về bản chất, Market-1501 không chỉ giúp mô hình học cách phân loại các ID trong tập train, mà còn giúp mô hình học một không gian embedding có khả năng tổng quát hóa. Sau khi huấn luyện, mô hình OSNet_x0_25 có thể được sử dụng để trích xuất đặc trưng cho người mới trong video thực nghiệm, dù người đó không nằm trong tập Market-1501. Đây là đặc điểm quan trọng của Person Re-ID.

Trong đề tài, Market-1501 được dùng để huấn luyện OSNet_x0_25 nhằm tạo ra bộ trích xuất đặc trưng ngoại hình. Sau khi mô hình đã học được cách biểu diễn người dưới dạng embedding vector, hệ thống có thể áp dụng mô hình này lên video thực tế. Khi đó, mỗi người được YOLO26n phát hiện sẽ được crop và đưa qua OSNet để tạo embedding. Các embedding này được so sánh với embedding của người mục tiêu để quyết định đối tượng cần bám theo.

2.7.2 Cấu trúc thư mục và các tập train/query/gallery

Market-1501 có cấu trúc dữ liệu được tổ chức để phục vụ cả huấn luyện và đánh giá mô hình Person Re-ID. Trong notebook, dữ liệu được liên kết vào thư mục:

```
reid-data/market1501/Market-1501-v15.09.15
```

Việc tạo symbolic link đến thư mục này giúp Torchreid nhận diện đúng dataset theo cấu trúc mà thư viện yêu cầu. Đây là bước chuẩn bị dữ liệu cần thiết trước khi khởi tạo ImageDataManager.

Cấu trúc phổ biến của Market-1501 gồm các thư mục chính sau:

```
Market-1501-v15.09.15
```

```
|— bounding_box_train
|— bounding_box_test
|— query
└— gt_bbox
```

Trong đó, thư mục `bounding_box_train` chứa các ảnh dùng để huấn luyện mô hình. Mỗi ảnh trong tập train có nhãn person ID tương ứng. Khi huấn luyện OSNet_x0_25 với softmax loss, mỗi person ID trong tập train được xem như một lớp phân loại. Mô hình học cách phân biệt các ID này, từ đó hình thành không gian đặc trưng cho bài toán Re-ID.

Thư mục `query` chứa các ảnh truy vấn. Trong quá trình đánh giá Re-ID, mỗi ảnh query được đưa vào mô hình để trích xuất embedding, sau đó so sánh với các ảnh trong tập gallery. Mục tiêu là kiểm tra xem mô hình có thể tìm lại đúng người trong gallery hay không. Tập query đại diện cho ảnh người cần tìm.

Thư mục `bounding_box_test` thường đóng vai trò là gallery. Gallery là tập ảnh chứa nhiều người khác nhau, trong đó có các ảnh cùng ID với ảnh query. Khi đánh giá, mô hình sẽ xếp hạng các ảnh trong gallery theo mức độ giống với ảnh query. Nếu các ảnh cùng người với query được xếp ở vị trí cao, mô hình được xem là có khả năng Re-ID tốt.

Có thể mô tả quá trình đánh giá query-gallery như sau. Với một ảnh query I_q , mô hình trích xuất vector:

$$f_q = F_\theta(I_q) \quad (4)$$

Với mỗi ảnh gallery I_g , mô hình trích xuất vector:

$$F_g = F_\theta(I_g) \quad (5)$$

Sau đó, hệ thống tính độ tương đồng giữa query và từng ảnh gallery:

$$\text{sim}(f_q, f_g) = \frac{f_q \cdot f_g}{\|f_q\| \times \|f_g\|} \quad (6)$$

Các ảnh gallery được sắp xếp theo độ tương đồng giảm dần. Nếu ảnh cùng person ID với query nằm ở các vị trí đầu trong danh sách, mô hình có khả năng tái nhận dạng tốt.

Trong notebook của đề tài, Torchreid quản lý các phần train, query và gallery thông qua ImageDataManager. Cấu hình sử dụng sources='market1501' và targets='market1501', nghĩa là mô hình được huấn luyện và đánh giá trên Market-1501. Kích thước ảnh được đặt là height=256 và width=128, phù hợp với chuẩn ảnh người trong bài toán Re-ID.

Việc hiểu rõ cấu trúc train/query/gallery giúp giải thích đúng vai trò của Market-1501. Tập train dùng để học tham số mô hình, còn query và gallery dùng để đánh giá khả năng tìm lại đúng người. Trong hệ thống thực tế của đề tài, cơ chế này được áp dụng theo hướng đơn giản hơn: ảnh người mục tiêu ban đầu đóng vai trò giống query, còn các người được phát hiện trong frame hiện tại đóng vai trò giống gallery nhỏ tại thời điểm đó.

2.7.3 Ý nghĩa của person ID và camera ID

Trong Market-1501, mỗi ảnh người thường đi kèm với hai loại nhãn quan trọng: person ID và camera ID. Person ID cho biết ảnh đó thuộc về người nào, còn camera ID cho biết ảnh được chụp từ camera nào. Hai nhãn này đóng vai trò quan trọng trong cả huấn luyện và đánh giá mô hình Person Re-ID.

Person ID là nhãn định danh của từng người trong tập dữ liệu. Các ảnh có cùng person ID được xem là thuộc cùng một người, dù chúng có thể được chụp ở các

thời điểm, góc nhìn hoặc camera khác nhau. Trong quá trình huấn luyện với softmax loss, mỗi person ID trong tập train được xem như một lớp phân loại. Mô hình học cách phân biệt các lớp này để tạo ra đặc trưng ngoại hình có tính phân biệt.

Nếu một ảnh huấn luyện được ký hiệu là I_i , person ID tương ứng là y_i , và tập huấn luyện có tổng cộng N person ID khác nhau, quá trình huấn luyện OSNet có thể được mô tả như một bài toán phân loại định danh người. Khi đó, mô hình không dự đoán trực tiếp nhãn từ ảnh đầu vào, mà trước hết trích xuất vector đặc trưng ngoại hình của ảnh người:

$$f_i = F_\theta(I_i) \quad (7)$$

Vector đặc trưng f_i sau đó được đưa qua lớp phân loại để tạo ra vector điểm số cho các person ID trong tập train:

$$z_i = C(f_i) \quad (8)$$

Từ vector điểm số z_i , hàm softmax được sử dụng để chuyển đổi thành xác suất dự đoán cho từng person ID:

$$\hat{p}_{i,k} = \frac{e^{z_{i,k}}}{\sum_{j=1}^N e^{z_{i,j}}}, k = 1, 2, 3, \dots, N \quad (9)$$

Nhãn dự đoán của mô hình được xác định là lớp có xác suất lớn nhất:

$$\hat{y}_i = \arg \max_k \hat{p}_{i,k} \quad (10)$$

Trong đó:

I_i : ảnh người đầu vào thứ i .

y_i : person ID thật của ảnh I_i

F_θ : mô hình trích xuất đặc trưng, trong đề tài là OSNet_x0_25.

θ : bộ tham số học được của mô hình.

f_i : embedding vector biểu diễn đặc trưng ngoại hình của ảnh người.

$C(\cdot)$: lớp phân loại dùng trong giai đoạn huấn luyện.

z_i : vector điểm số phân loại trước softmax.

$\hat{p}_{i,k}$: xác suất mô hình dự đoán ảnh I_i thuộc person ID thứ k .

$\hat{y}_i$: person ID được mô hình dự đoán.

N : số lượng person ID trong tập huấn luyện.

Trong quá trình huấn luyện với softmax loss, mục tiêu của mô hình là làm cho xác suất tại đúng nhãn y_i càng lớn càng tốt. Hàm mất mát cho một ảnh huấn luyện có thể được viết dưới dạng:

$$L_i = -\log(\hat{p}_{i,y_i}) \quad (10)$$

Công thức trên cho thấy nếu mô hình dự đoán đúng person ID với xác suất cao thì giá trị mất mát nhỏ; ngược lại, nếu xác suất tại nhãn thật thấp thì giá trị mất mát tăng lên. Nhờ quá trình tối ưu này, OSNet học được không gian đặc trưng trong đó ảnh của cùng một người có xu hướng nằm gần nhau, còn ảnh của những người khác nhau có xu hướng tách xa nhau.

Tuy nhiên, sau khi huấn luyện, hệ thống trong đề tài không dùng OSNet để phân loại người thực tế thành một ID cố định trong Market-1501. Lớp phân loại $C(\cdot)$ chủ yếu phục vụ giai đoạn huấn luyện. Khi triển khai trên video thực tế của xe, hệ thống sử dụng phần trích xuất đặc trưng F_θ để tạo embedding vector cho ảnh người, sau đó so sánh các embedding bằng cosine similarity. Vì vậy, OSNet được dùng như một bộ trích xuất đặc trưng ngoại hình, không phải bộ nhận diện tên hoặc ID thật của người dùng.

Camera ID cho biết ảnh được thu từ camera nào. Trong Re-ID, thông tin camera ID rất quan trọng vì cùng một người có thể xuất hiện dưới các góc nhìn khác nhau ở các camera khác nhau. Một mô hình Re-ID tốt cần nhận ra cùng một người dù ảnh được chụp từ camera khác, với nền cảnh, độ sáng và hướng nhìn khác nhau. Do đó, camera ID giúp quá trình đánh giá phản ánh đúng bản chất của bài toán tái nhận dạng người.

Trong quá trình đánh giá, nếu một ảnh query và một ảnh gallery có cùng person ID nhưng khác camera ID, đó thường là một cặp ảnh đúng cần được mô hình nhận ra. Ngược lại, nếu hai ảnh có person ID khác nhau, mô hình cần phân biệt chúng

là hai người khác nhau. Điều này buộc mô hình phải học đặc trưng ngoại hình của người thay vì chỉ ghi nhớ nền ảnh hoặc đặc điểm của một camera cố định.

Trong ngữ cảnh đề tài, person ID giúp OSNet_x0_25 học cách phân biệt người này với người khác trong quá trình huấn luyện. Camera ID giúp mô hình tiếp xúc với sự thay đổi góc nhìn và điều kiện quan sát. Nhờ đó, khi áp dụng vào video thực tế của xe, mô hình có khả năng tạo embedding ổn định hơn cho người mục tiêu, dù người đó thay đổi tư thế hoặc khoảng cách so với camera.

Tuy nhiên, cần lưu ý rằng person ID trong Market-1501 không phải là ID của người dùng trong hệ thống thực tế. Khi xe hoạt động, người mục tiêu có thể là một người hoàn toàn mới, không xuất hiện trong dataset. Do đó, hệ thống không dùng OSNet để dự đoán “người này là ID số bao nhiêu trong Market-1501”, mà dùng OSNet để tạo vector đặc trưng và so sánh mức độ giống nhau giữa các ảnh người.

2.7.4 Vai trò của Market-1501 trong đề tài

Market-1501 đóng vai trò là tập dữ liệu huấn luyện nền cho mô hình OSNet_x0_25. Nhờ dataset này, mô hình học được cách trích xuất đặc trưng ngoại hình của người từ ảnh crop. Những đặc trưng này sau đó được sử dụng trong hệ thống xe tự hành để so khớp người mục tiêu qua các frame video. Nói cách khác, Market-1501 giúp mô hình học khả năng phân biệt người trước khi được đưa vào pipeline thực tế.

Trong đề tài, Market-1501 không được sử dụng để huấn luyện mô hình phát hiện người YOLO26n. YOLO26n được nạp từ trọng số pretrained và dùng để phát hiện các đối tượng thuộc lớp người. Sau đó, các vùng người được YOLO phát hiện mới được đưa vào OSNet_x0_25. Vì vậy, vai trò của Market-1501 nằm ở tầng Re-ID, không nằm ở tầng detection. Điểm này cần được trình bày rõ để tránh nhầm lẫn giữa dữ liệu huấn luyện YOLO và dữ liệu huấn luyện OSNet.

Trong notebook, mô hình OSNet_x0_25 được huấn luyện trên Market-1501 với các cấu hình như `batch_size_train=32`, `batch_size_test=100`, kích thước ảnh 256 x 128, augmentation gồm `random_flip` và `random_crop`, optimizer Adam với learning rate 0.0015, scheduler `single_step`, label smoothing và số epoch huấn

luyện là 30. Sau khi huấn luyện, checkpoint của mô hình được nạp lại để phục vụ suy luận và export sang ONNX .

Vai trò thực tế của Market-1501 trong hệ thống có thể được mô tả như sau:

Market-1501

- Huấn luyện OSNet_x0_25
- Tạo mô hình trích xuất embedding người
- Áp dụng lên crop người từ YOLO26n
- So khớp người mục tiêu bằng cosine similarity

Nhờ được huấn luyện trên Market-1501, OSNet_x0_25 có khả năng biến ảnh người thành embedding vector có ý nghĩa. Trong video thực nghiệm, khi người mục tiêu được khóa ở frame đầu tiên, hệ thống lưu lại embedding của người đó. Ở các frame tiếp theo, mọi người được YOLO26n phát hiện sẽ được đưa qua OSNet để tạo embedding mới. Các embedding này được so sánh với embedding mục tiêu để xác định người cần bám theo.

Một lợi ích của việc sử dụng Market-1501 là dataset chứa nhiều người và nhiều góc nhìn khác nhau. Điều này giúp mô hình học được các đặc trưng tổng quát hơn so với việc chỉ train trên một số ít ảnh tự thu thập. Nếu chỉ dùng dữ liệu tự quay với số lượng nhỏ, mô hình có thể dễ bị overfitting và khó hoạt động ổn định khi môi trường thay đổi. Market-1501 cung cấp nền tảng dữ liệu tốt hơn cho giai đoạn huấn luyện ban đầu.

Tuy nhiên, Market-1501 cũng có giới hạn khi áp dụng vào đề tài. Dataset này được thu trong môi trường camera giám sát, trong khi hệ thống xe sử dụng camera di động gắn trên thân xe. Góc nhìn, độ rung, khoảng cách và điều kiện ánh sáng trong hệ thống thực tế có thể khác với dữ liệu train. Đây được gọi là khoảng cách miền dữ liệu giữa dataset huấn luyện và môi trường triển khai. Vì vậy, sau khi huấn luyện trên Market-1501, hệ thống vẫn cần được kiểm thử bằng video thực nghiệm để đánh giá khả năng bám đúng người trong điều kiện thực tế.

Tóm lại, Market-1501 là tập dữ liệu quan trọng giúp huấn luyện mô hình OSNet_x0_25 cho bài toán Person Re-ID. Dataset này cung cấp ảnh người có nhãn person ID và camera ID, giúp mô hình học cách phân biệt người dựa trên đặc trưng

ngoại hình. Trong đề tài, Market-1501 đóng vai trò nền tảng cho tầng tái nhận dạng người, hỗ trợ hệ thống xác định đúng người mục tiêu sau khi YOLO26n phát hiện các ứng viên trong frame.

Chương 3

THIẾT KẾ HỆ THỐNG

3.1 TỔNG QUAN THIẾT KẾ HỆ THỐNG

Mục tiêu của đề tài là tạo ra một hệ thống xe tự hành thông minh có khả năng nhận diện, khóa và bám sát một mục tiêu cụ thể trong thời gian thực. Bằng việc ứng dụng công nghệ thị giác máy tính và các thuật toán học sâu, hệ thống có thể liên tục theo dõi quỹ đạo, duy trì hướng di chuyển để giữ mục tiêu luôn ở trung tâm khung hình, đồng thời có khả năng tự động đổi chiều đặc trưng ngoại hình để không bị đánh lừa bởi những yếu tố nhiễu (người lạ đi ngang qua, vật cản tạm thời).

3.1.1 Mục tiêu thiết kế

Để hiện thực hóa mục tiêu trên, mô hình hoạt động của xe được thiết kế dựa trên nguyên lý điều khiển vòng kín với luồng xử lý tín hiệu liên tục. Hệ thống có các khối chính như sau:

Khối thu thập hình ảnh (Camera): Đóng vai trò là thiết bị cảm biến tầm nhìn, có nhiệm vụ liên tục quay quét môi trường thực tế phía trước xe và truyền luồng dữ liệu hình ảnh (video stream) về mạch điều khiển với độ trễ thấp nhất để làm đầu vào phân tích.

Khối xử lý trung tâm (Máy tính nhúng): Đóng vai trò là "bộ não" điều phối toàn bộ hệ thống. Khối này đảm nhiệm việc chạy các mạng nơ-ron nhân tạo để phát hiện con người, trích xuất "vân tay ngoại hình", và tính toán độ tương đồng (thuật toán Jaccard/Cosine) để chốt đúng tọa độ mục tiêu VIP. Dựa vào tọa độ đó, hệ thống sẽ tính toán sai số (lệch góc, lệch khoảng cách) và xuất ra các tín hiệu điều khiển số (GPIO, PWM).

Khối điều khiển (Motor Driver): Làm cầu nối trung gian giữa mạch logic và cơ cấu chấp hành. Nó làm nhiệm vụ cách ly quang học để bảo vệ mạch trung tâm, đồng thời nhận tín hiệu điện áp thấp để băm xung, đóng/cắt dòng điện lớn và điện áp cao cấp xuống cho động cơ.

Khối cơ cấu chấp hành (Động cơ DC): Tiếp nhận dòng điện từ mạch điều khiển để chuyển hóa điện năng thành cơ năng. Động cơ thực thi trực tiếp các lệnh vật lý như tăng tốc, giảm tốc, hoặc chạy vi sai (bánh trái/phải lệch tốc) để điều hướng xe di chuyển bám theo mục tiêu.

Khối nguồn (Power Supply): Cung cấp và phân phối năng lượng cho toàn bộ mô hình. Khối này sẽ phân tách điện áp thành các mức độc lập: nguồn ổn áp dòng thấp cho hệ thống xử lý logic và nguồn công suất lớn cho hệ thống động cơ tải nặng, đảm bảo mạch không bị sụt áp hay nhiễu điện từ trường khi hoạt động.

3.1.2 Yêu cầu chức năng

Hệ thống xe tự hành bám sát mục tiêu được thiết kế với hai chế độ hoạt động chính là chế độ điều khiển thủ công và chế độ tự động. Việc phân chia thành hai chế độ giúp hệ thống vừa có khả năng vận hành theo lệnh trực tiếp từ người dùng, vừa có khả năng tự xử lý hình ảnh và điều khiển chuyển động dựa trên thuật toán trí tuệ

nhân tạo.

a. Chế độ điều khiển thủ công

Ở chế độ điều khiển thủ công, người dùng trực tiếp điều khiển hướng di chuyển và tốc độ của xe thông qua các nút bấm trên ứng dụng Flutter. Giao diện điều khiển được bố trí các nút chức năng tương ứng với các hướng di chuyển cơ bản như tiến, lùi, rẽ trái và rẽ phải. Khi người dùng nhấn một nút điều khiển, ứng dụng sẽ xác định hướng di chuyển tương ứng và chuyển đổi thành các thông số điều khiển gồm chiều quay động cơ trái, chiều quay động cơ phải và tốc độ di chuyển của xe.

Các thông số điều khiển này được biểu diễn dưới dạng dirL, dirR và speed, trong đó dirL là chiều quay của động cơ bên trái, dirR là chiều quay của động cơ bên phải, còn speed là giá trị tốc độ mong muốn. Ví dụ, khi người dùng nhấn nút tiến, hai động cơ trái và phải cùng quay theo chiều tiến; khi nhấn nút lùi, hai động

cơ cùng quay theo chiều ngược lại; khi nhấn nút rẽ trái hoặc rẽ phải, hệ thống điều chỉnh chiều quay hoặc tốc độ giữa hai động cơ để tạo chuyển động quay của xe.

Sau khi được tạo ra, các thông số điều khiển được gửi từ ứng dụng Flutter đến Raspberry Pi thông qua giao tiếp Socket.IO. Raspberry Pi tiếp nhận dữ liệu điều khiển, sau đó truyền lệnh xuống vi điều khiển thông qua giao tiếp UART để điều khiển động cơ. Trong chế độ này, xe có thể thực hiện các trạng thái chuyển động cơ bản như tiến, lùi, rẽ trái, rẽ phải và dừng.

Khi người dùng không nhấn nút điều khiển hoặc nhấn nút dừng, hệ thống sẽ gửi lệnh dừng nhằm đảm bảo xe không di chuyển ngoài ý muốn. Chế độ thủ công giúp người dùng dễ dàng kiểm tra phần cứng, thử nghiệm khả năng điều khiển động cơ và can thiệp trực tiếp khi cần thiết.

b. Chế độ tự động

Ở chế độ tự động, Raspberry Pi đóng vai trò là bộ xử lý trung tâm của hệ thống. Camera được sử dụng để thu nhận hình ảnh liên tục từ môi trường phía trước xe. Các khung hình thu được sẽ được đưa vào mô hình YOLO để phát hiện các đối tượng người xuất hiện trong ảnh. Kết quả phát hiện bao gồm vị trí bounding box và độ tin cậy của từng đối tượng.

Sau khi phát hiện người trong khung hình, hệ thống sử dụng mô hình OSNet để trích xuất đặc trưng nhận dạng của từng đối tượng. Các đặc trưng này được dùng để so sánh và tái nhận dạng người mục tiêu, giúp hệ thống duy trì đúng đối tượng cần bám theo ngay cả khi trong khung hình có nhiều người xuất hiện. Khi đã xác định được người mục tiêu, Raspberry Pi tính toán các thông số bám theo như sai lệch vị trí ngang của mục tiêu so với tâm khung hình, kích thước bounding box hiện tại và tỉ lệ khoảng cách tương đối.

Dựa trên các thông số này, Raspberry Pi tạo lệnh tracking và gửi xuống vi điều khiển thông qua giao tiếp UART. Vi điều khiển tiếp nhận lệnh, sau đó điều khiển động cơ trái và phải để xe di chuyển theo hướng phù hợp. Nếu mục tiêu nằm lệch trái hoặc lệch phải so với tâm ảnh, xe sẽ điều chỉnh hướng di chuyển tương ứng. Nếu mục tiêu ở quá xa, xe tiến lại gần; nếu mục tiêu quá gần hoặc bị mất khỏi khung hình, hệ thống sẽ dừng hoặc chuyển sang trạng thái chờ tìm lại mục tiêu.

3.2 KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

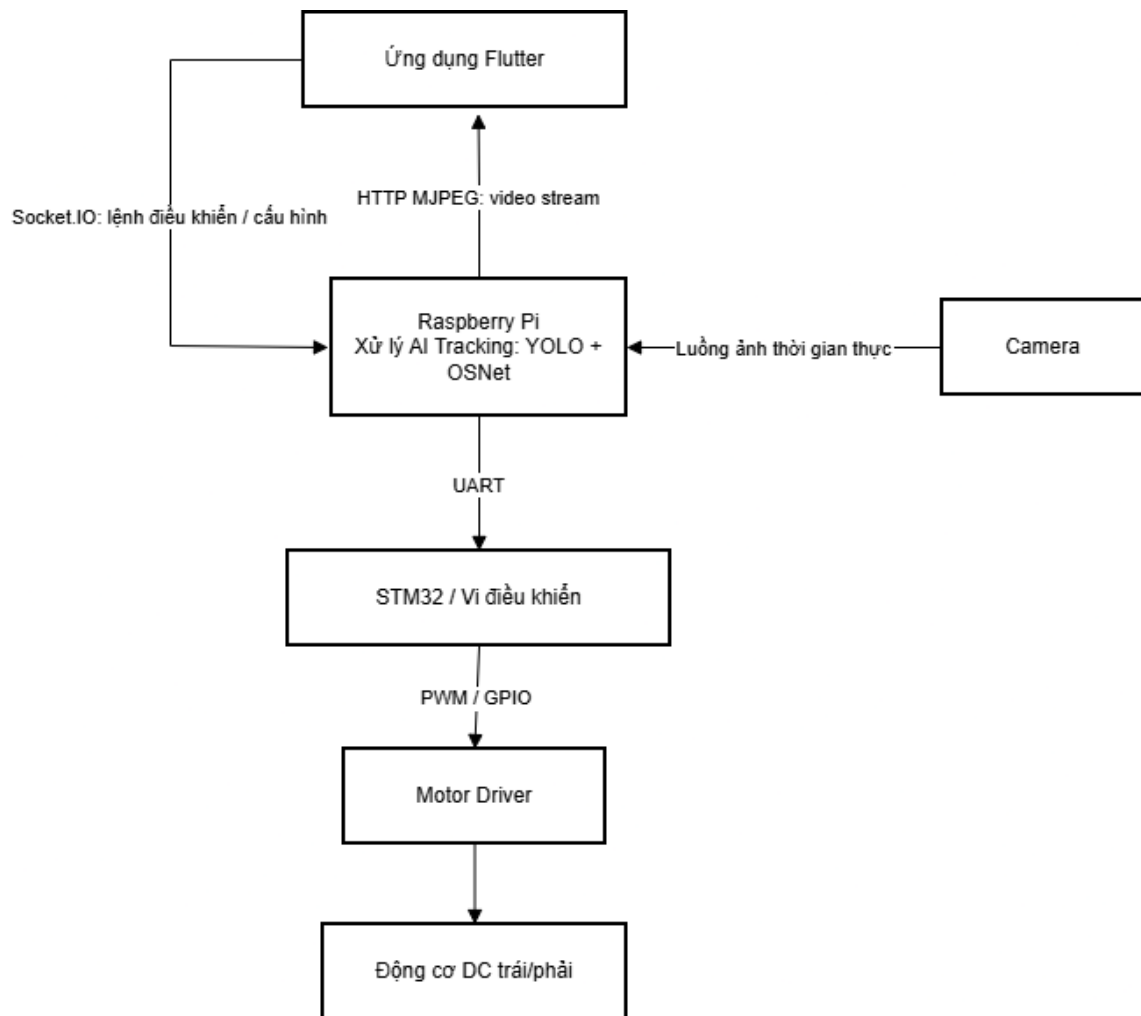
Kiến trúc tổng thể của hệ thống được thiết kế theo hướng phân tách rõ ràng giữa khối giao diện điều khiển, khối xử lý trung tâm và khối chấp hành. Trong đó, ứng dụng Flutter đóng vai trò là giao diện người dùng, cho phép quan sát hình ảnh từ camera, chuyển đổi chế độ hoạt động và gửi lệnh điều khiển đến xe. Raspberry Pi đóng vai trò là bộ xử lý trung tâm, thực hiện các nhiệm vụ như cung cấp luồng video, nhận lệnh từ ứng dụng, xử lý thuật toán AI tracking và truyền lệnh điều khiển xuống vi điều khiển. Vi điều khiển tiếp nhận lệnh từ Raspberry Pi, sau đó điều khiển motor driver để tạo chuyển động cho động cơ DC.

3.2.1 Sơ đồ khối tổng thể

Sơ đồ khối tổng thể của hệ thống được xây dựng nhằm thể hiện mối quan hệ giữa các thành phần chính, bao gồm ứng dụng điều khiển Flutter, Raspberry Pi, camera, vi điều khiển, motor driver và động cơ DC. Camera được kết nối trực tiếp với Raspberry Pi để cung cấp dữ liệu hình ảnh thời gian thực. Dữ liệu hình ảnh này vừa được Raspberry Pi xử lý bằng thuật toán AI tracking, vừa được đóng gói thành luồng video MJPEG để truyền về ứng dụng Flutter.

Ứng dụng Flutter có nhiệm vụ hiển thị luồng camera, cung cấp điều khiển thủ công và cho phép người dùng cấu hình chế độ hoạt động của xe. Khi người dùng chọn chế độ Manual, lệnh điều khiển từ ứng dụng Flutter được gửi đến Raspberry Pi thông qua Socket.IO. Khi người dùng chọn chế độ Auto, Raspberry Pi sử dụng dữ liệu camera để phát hiện và bám theo mục tiêu, sau đó gửi thông số điều khiển xuống vi điều khiển thông qua UART.

Vi điều khiển nhận dữ liệu từ Raspberry Pi, xử lý lệnh điều khiển và xuất tín hiệu PWM/GPIO đến motor driver. Motor driver có nhiệm vụ khuếch đại tín hiệu điều khiển để cung cấp đủ dòng và điện áp cho động cơ DC. Cuối cùng, động cơ DC tạo ra chuyển động thực tế cho xe, bao gồm tiến, lùi, rẽ trái, rẽ phải hoặc dừng.



Hình 3.1 Sơ đồ khối tổng thể của hệ thống

3.2.2 Chức năng các khối trong hệ thống

Hệ thống được chia thành nhiều khối chức năng nhằm đảm bảo mỗi thành phần đảm nhiệm một nhiệm vụ cụ thể.

Bảng 3.1 Chức năng các khối chính trong hệ thống

Khối chức năng	Chức năng
Camera	Thu nhận hình ảnh phía trước xe theo thời gian thực. Dữ liệu hình ảnh được gửi về Raspberry Pi để phục vụ quá trình hiển thị video và xử lý AI tracking.

Raspberry Pi	Đóng vai trò là bộ xử lý trung tâm của hệ thống. Raspberry Pi chạy server, cung cấp luồng video MJPEG, nhận lệnh điều khiển từ ứng dụng Flutter thông qua Socket.IO, thực hiện thuật toán phát hiện và bám theo mục tiêu bằng AI, sau đó truyền lệnh điều khiển xuống vi điều khiển thông qua UART.
Ứng dụng Flutter	Là giao diện điều khiển của người dùng. Ứng dụng có nhiệm vụ hiển thị luồng camera, cung cấp điều khiển thủ công, cho phép chuyển đổi giữa chế độ Manual và Auto, đồng thời hỗ trợ cài đặt các tham số AI như mô hình YOLO và ngưỡng confidence.
STM32 / Vi điều khiển	Nhận lệnh điều khiển từ Raspberry Pi thông qua UART. Sau khi nhận dữ liệu, vi điều khiển xử lý lệnh và xuất tín hiệu điều khiển phù hợp đến motor driver để điều khiển chuyển động của xe.
Motor Driver	Đóng vai trò là tầng công suất trung gian giữa vi điều khiển và động cơ DC. Motor driver nhận tín hiệu điều khiển từ vi điều khiển, sau đó khuếch đại dòng và điện áp để điều khiển chiều quay cũng như tốc độ của động cơ.
Động cơ DC	Là cơ cấu chấp hành chính của hệ thống, tạo ra chuyển động thực tế cho xe. Hai động cơ DC trái và phải được điều khiển độc lập để xe có thể tiến, lùi, rẽ trái, rẽ phải hoặc dừng.

3.3 THIẾT KẾ PHẦN CỨNG

3.3.1 Chức năng của phần cứng

Để hiện thực hóa mô hình xe tự lái đa chế độ và đáp ứng trọn vẹn 4 mục tiêu chức năng cốt lõi đã đề ra ở Chương 1, hệ thống phần cứng được thiết kế với các chức năng chi tiết sau:

Nền tảng xử lý và điều khiển trung tâm: Đảm nhiệm vai trò bộ não của toàn hệ thống. Phần cứng này phải có khả năng tính toán mạnh mẽ để chạy các thuật toán tối ưu hóa của mạng nơ-ron OSNet, nhưng các mô hình AI để xử lý hình ảnh

thời gian thực, quản lý logic chuyển đổi giữa các chế độ lái, tính toán sai số làn đường và xuất tín hiệu điều khiển xuống cơ cấu chấp hành.

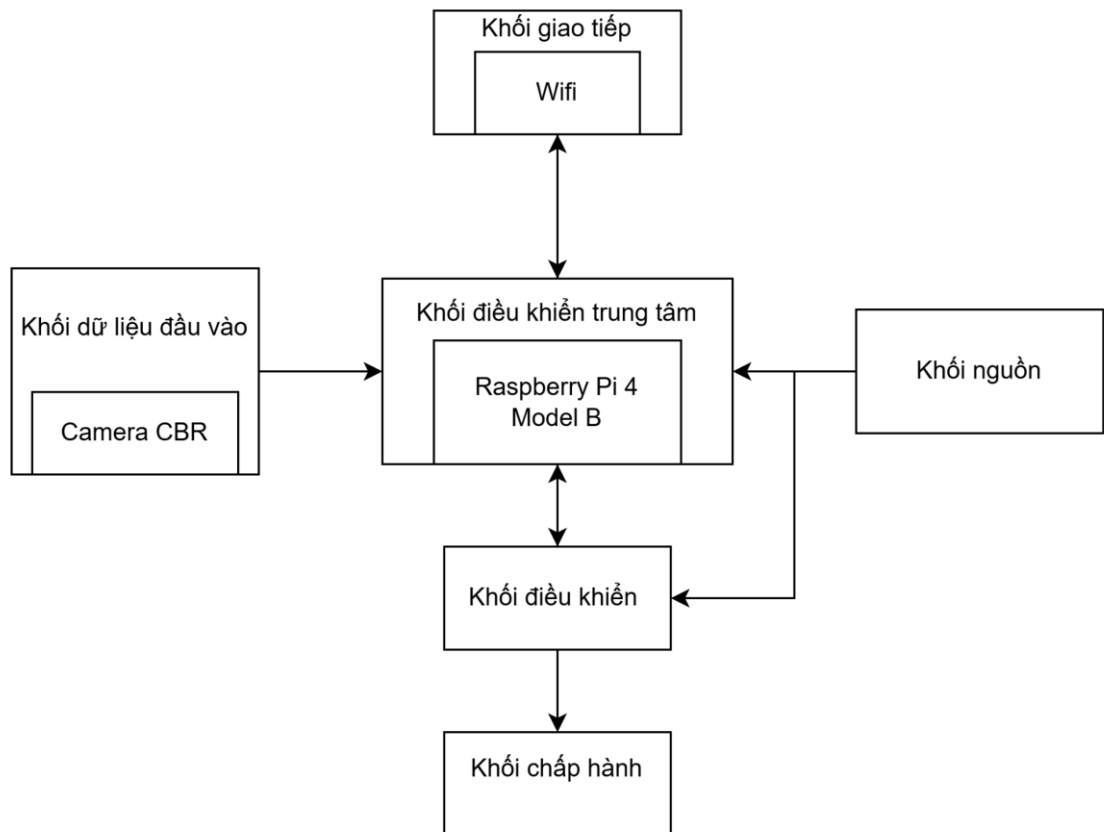
Thu thập dữ liệu thị giác: Đóng vai trò là đôi mắt, liên tục thu thập luồng dữ liệu hình ảnh thời gian thực từ môi trường sa bàn. Dữ liệu này là đầu vào duy nhất cho Khối xử lý trung tâm để thực hiện khả năng theo dõi mục tiêu trọng điểm ở frame 1.

Giao tiếp không dây điều khiển thủ công: Cung cấp kênh truyền tải dữ liệu không dây công suất thấp, độ trễ thấp để kết nối trực tiếp với ứng dụng trên điện thoại thông minh. Chức năng này phục vụ riêng cho việc quản lý và kiểm soát thiết bị khi cần thiết.

Khuếch đại công suất và cách ly điều khiển: Đóng vai trò cầu nối giữa tín hiệu điều khiển điện áp thấp từ mạch trung tâm và hệ thống động cơ DC tải nặng. Chức năng này bao gồm việc cách ly quang học để bảo vệ mạch xử lý nhạy cảm, thực hiện băm xung PWM để điều chỉnh tốc độ, và đảo chiều dòng điện cấp cho động cơ để điều khiển hướng di chuyển của xe.

Cấp nguồn ổn định và cách ly: Phân phối và ổn áp năng lượng từ pin tổng thành các cấp điện áp độc lập: nguồn sạch 5V dòng cao cho mạch logic xử lý AI và nguồn công suất tải nặng cho động cơ. Chức năng này đảm bảo hệ thống không bị treo hay nhiễu khi động cơ tăng tốc đột ngột đuổi theo mục tiêu.

3.3.2 Sơ đồ khối phần cứng



Hình 3.2 Sơ đồ khối cho hệ thống

3.3.3 Thiết kế từng khối

a) Khối Xử lý trung tâm (Raspberry Pi 4)

Bo mạch Raspberry Pi 4 kết nối với camera qua cổng CSI hoặc USB, đảm nhận toàn bộ tác vụ xử lý AI. Giao tiếp với khối điều khiển động học phía dưới được thực hiện qua chuẩn UART ở tốc độ 115.200 bps.

Trong quá trình hoạt động, Pi liên tục thu nhận hình ảnh, chạy mô hình YOLO để phát hiện người và OSNet để trích xuất đặc trưng nhận dạng. Sau khi tính toán sai số vị trí mục tiêu, kết quả được đóng gói thành bản tin điều khiển và truyền xuống STM32.

b) Khối Điều khiển động học (STM32F103C8T6)

Vi điều khiển STM32F103C8T6 (Blue Pill) đảm nhận vai trò xử lý thời gian thực cho hệ thống truyền động. Bo mạch sử dụng thạch anh ngoại 8 MHz, nhân

lên 72 MHz qua PLL nội, đảm bảo độ chính xác cao cho xung PWM và giao tiếp UART.

Khi nhận lệnh từ Pi qua ngắt UART, STM32 trực tiếp điều khiển chiều quay động cơ qua GPIO và điều chỉnh tốc độ qua Hardware Timer PWM. Việc tách riêng khối này khỏi Raspberry Pi giúp xe di chuyển trơn tru, tránh hiện tượng giật xung (jitter) vốn thường gặp trên các hệ điều hành đa nhiệm như Linux.

c) Khối Chấp hành công suất (L298N)

Mạch cầu H L298N được sử dụng để dẫn động hai động cơ DC giảm tốc. Các diode Schottky được bổ sung tại đầu ra để bảo vệ IC khỏi dòng điện tự cảm sinh ra khi động cơ dừng đột ngột.

IC nhận tín hiệu logic điều hướng (IN1–IN4) và xung PWM điều tốc (ENA, ENB) từ STM32, cho phép xe thực hiện các chuyển động cơ bản: tiến, lùi và xoay tại chỗ.

d) Khối Nguồn cấp (Power Tree)

Nguồn chính là pin Li-Po 3S (danh định 11,1V, tối đa 12,6V). Để tránh nhiễu lẫn nhau giữa các khối, hệ thống chia thành ba nhánh cấp nguồn độc lập:

Nhánh công suất: Cấp thẳng 11,1V từ pin trực tiếp cho động cơ qua L298N.

Nhánh AI: Hạ áp qua mạch Buck XL4015 xuống 5,1V – 3A để nuôi Raspberry Pi ổn định.

Nhánh logic: Lấy nguồn 5V từ chân ra của Raspberry Pi để cấp cho STM32 và các cảm biến.

3.3.4 Tính toán thông số linh kiện và tổng công suất

Dưới đây là bảng liệt kê thông số tiêu thụ điện năng của các linh kiện chính trong điều kiện tải nặng nhất (Worst-case scenario):

Bảng 3.2 Liệt kê thông số tiêu thụ điện năng của các linh kiện chính

STT	Tên linh kiện	Điện áp hoạt động (V)	Dòng điện tối đa (A)	Công suất tối đa (W)
-----	---------------	-----------------------	----------------------	----------------------

1	Raspberry Pi 4 (bao gồm Camera)	5.1	3.00	15.30
2	Động cơ DC giảm tốc (Tổng 4 động cơ)	12.0	4.8	57.6
3	Vi điều khiển STM32F103C8T6	3.3	0.10	0.33
4	IC Cầu H L298N (Tổng hao nội tại)	12.0	0.05	0.60
TỔNG CÔNG SUẤT TOÀN HỆ THỐNG (W)				73.83

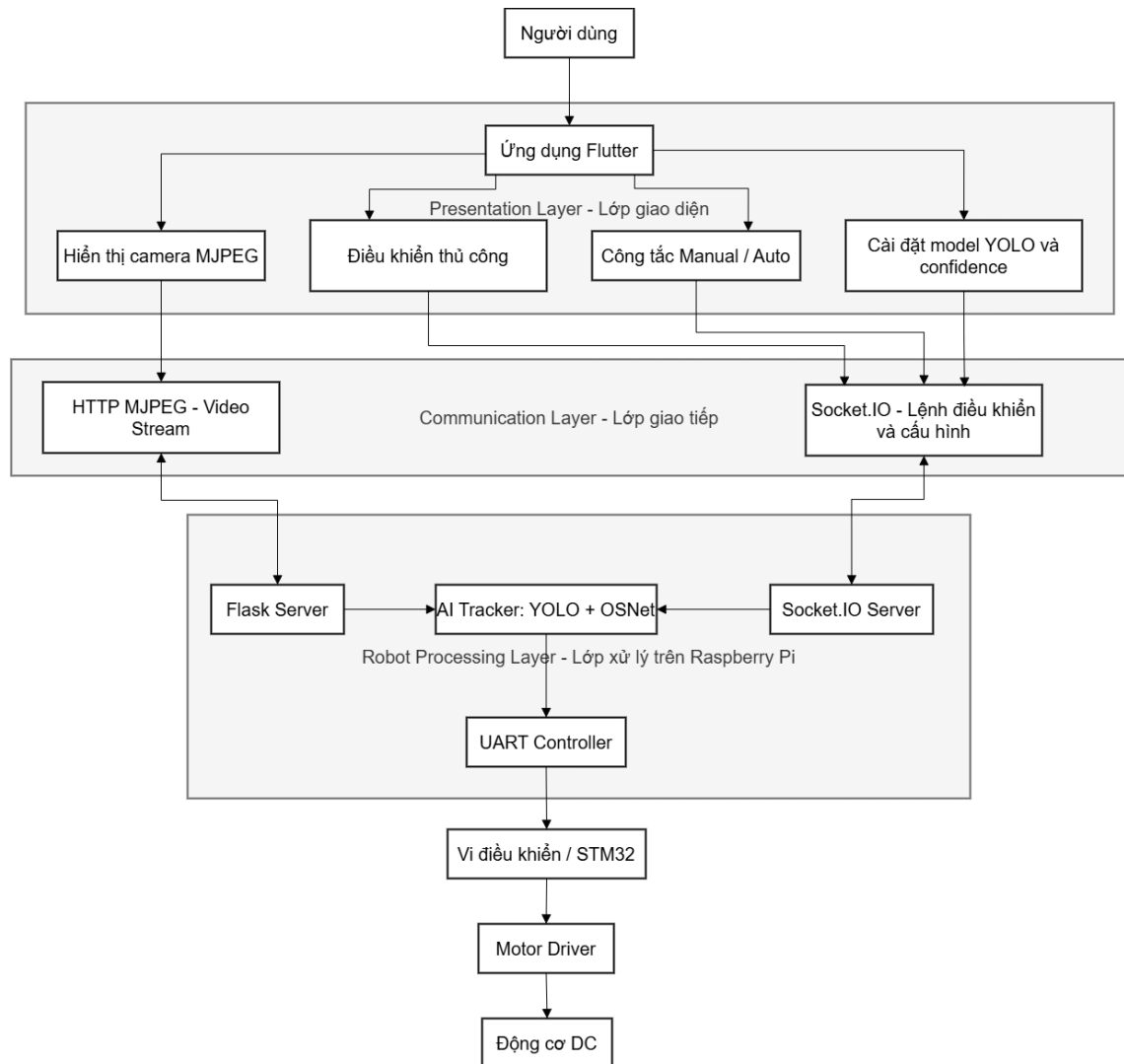
3.4 THIẾT KẾ PHẦN MỀM

Phần mềm của hệ thống được thiết kế nhằm đáp ứng hai yêu cầu chính: hỗ trợ người dùng điều khiển xe thủ công thông qua ứng dụng và cho phép xe hoạt động tự động dựa trên thuật toán thị giác máy tính. Để đạt được mục tiêu này, phần mềm được chia thành nhiều module chức năng, bao gồm ứng dụng điều khiển Flutter, server xử lý trên Raspberry Pi, module AI tracking và module giao tiếp UART với vi điều khiển.

3.4.1 Kiến trúc phần mềm

Kiến trúc phần mềm của hệ thống được xây dựng theo mô hình phân lớp, trong đó mỗi lớp đảm nhiệm một nhóm chức năng riêng. Ở mức tổng quát, phần mềm được chia thành ba lớp chính: lớp giao diện người dùng, lớp giao tiếp và lớp xử lý robot.

Lớp giao diện người dùng được triển khai bằng Flutter, có nhiệm vụ hiển thị luồng camera từ Raspberry Pi, cung cấp điều khiển thủ công, công tắc chuyển đổi chế độ Manual/Auto và giao diện cấu hình thông số AI. Lớp giao tiếp sử dụng HTTP MJPEG để truyền video từ Raspberry Pi về ứng dụng và Socket.IO để truyền các lệnh điều khiển theo thời gian thực. Lớp xử lý robot chạy trên Raspberry Pi, bao gồm server Flask/Socket.IO, module AI tracking và module UART để gửi lệnh xuống vi điều khiển.

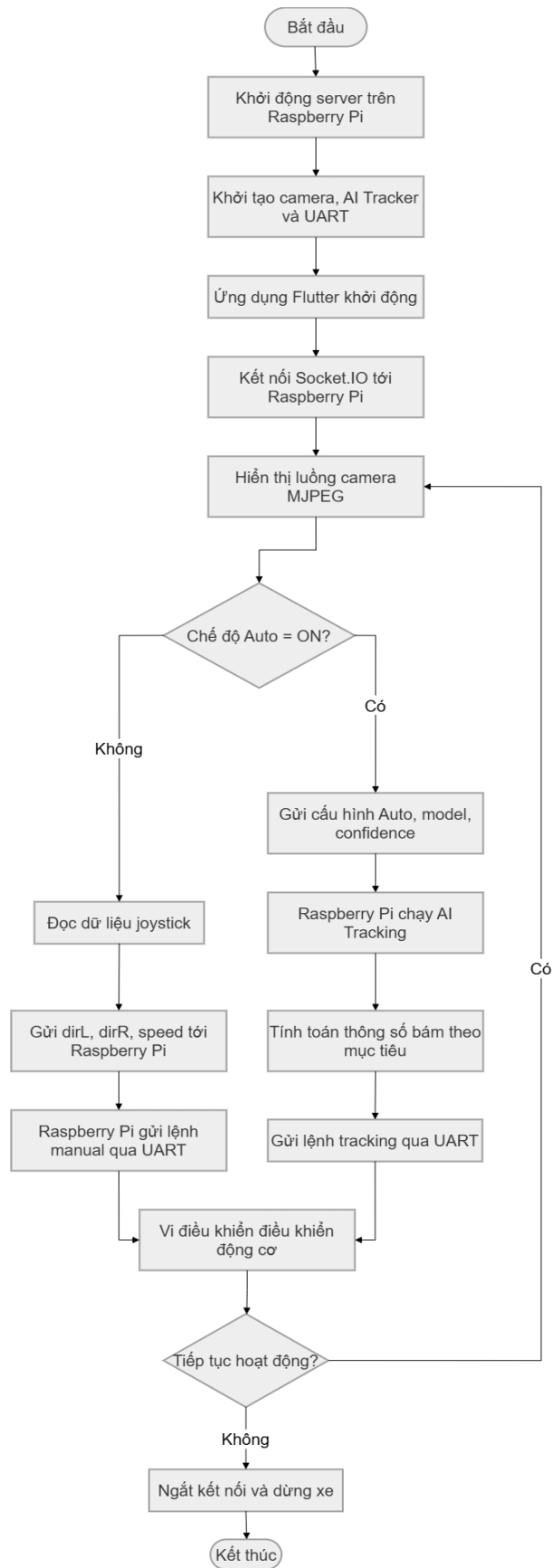


Hình 3.3 Sơ đồ kiến trúc phần mềm tổng quát

Trong kiến trúc này, ứng dụng Flutter không trực tiếp điều khiển động cơ mà chỉ gửi lệnh hoặc cấu hình đến Raspberry Pi. Raspberry Pi là thành phần trung gian chịu trách nhiệm xử lý lệnh, xử lý hình ảnh và gửi dữ liệu điều khiển xuống vi điều khiển. Cách thiết kế này giúp giảm độ phức tạp cho ứng dụng di động, đồng thời tập trung các tác vụ tính toán nặng như xử lý ảnh và chạy mô hình AI tại Raspberry Pi.

Về luồng hoạt động, khi hệ thống được khởi động, Raspberry Pi chạy server và sẵn sàng cung cấp luồng video. Ứng dụng Flutter kết nối đến Raspberry Pi thông qua Socket.IO, đồng thời nhận video từ endpoint MJPEG để hiển thị trên giao diện. Người dùng có thể chọn một trong hai chế độ hoạt động. Nếu chọn chế độ Manual,

ứng dụng gửi dữ liệu nút điều khiển đến Raspberry Pi. Nếu chọn chế độ Auto, Raspberry Pi sử dụng camera và mô hình AI để tự động xác định mục tiêu và sinh lệnh điều khiển.



Hình 3.4 Lưu đồ hoạt động tổng quát của phần mềm

3.4.2 Thiết kế ứng dụng điều khiển Flutter

Ứng dụng điều khiển được xây dựng bằng Flutter nhằm tạo giao diện trực quan, có khả năng chạy trên nhiều nền tảng như Android, iOS, Windows hoặc Web. Trong phạm vi đề tài, ứng dụng được sử dụng để điều khiển xe từ xa, quan sát luồng camera và cấu hình chế độ hoạt động của hệ thống.

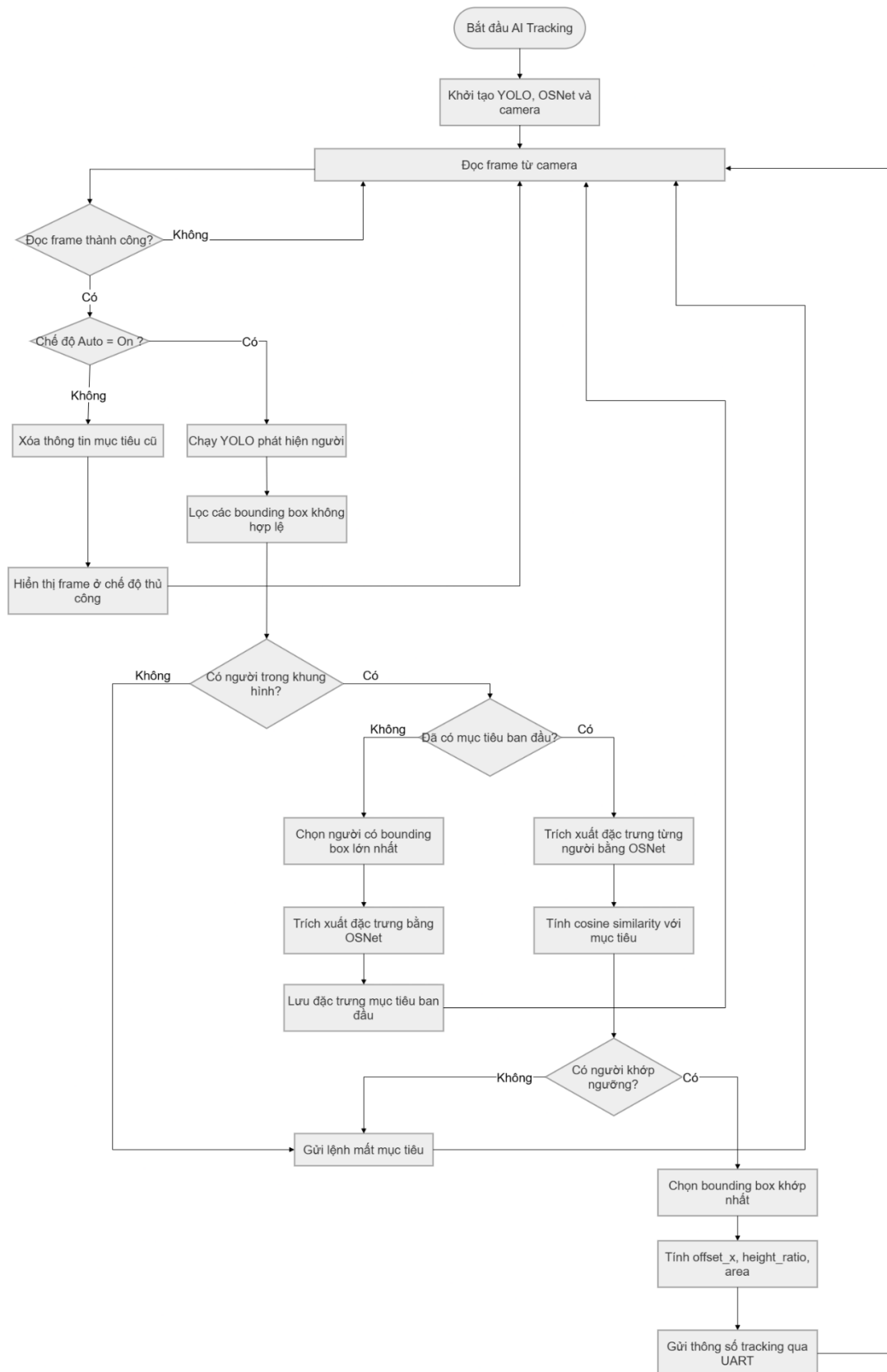
Cấu trúc chính của ứng dụng Flutter gồm ba thành phần quan trọng: file khởi chạy ứng dụng, màn hình điều khiển chính và lớp dịch vụ giao tiếp Socket.IO. File `main.dart` là điểm bắt đầu của chương trình, có nhiệm vụ khởi chạy ứng dụng và thiết lập màn hình chính. File `home_screen.dart` chứa giao diện điều khiển, xử lý trạng thái Manual/Auto, hiển thị camera, nút điều khiển và bảng cài đặt AI. File `socket_service.dart` đóng gói phần giao tiếp giữa ứng dụng và Raspberry Pi.

3.5 THIẾT KẾ THUẬT TOÁN AI TRACKING

Thuật toán AI Tracking là thành phần trung tâm trong chế độ tự động của hệ thống. Mục tiêu của thuật toán là giúp xe có khả năng phát hiện người trong khung hình, xác định đúng người mục tiêu cần bám theo, duy trì khả năng theo dõi mục tiêu qua nhiều khung hình và sinh ra các thông số điều khiển phù hợp để truyền xuống vi điều khiển.

Trong hệ thống này, thuật toán AI Tracking được triển khai trên Raspberry Pi. Camera cung cấp luồng ảnh đầu vào theo thời gian thực. Mô hình YOLO được sử dụng để phát hiện các đối tượng người trong ảnh, trong khi mô hình OSNet được sử dụng để trích xuất đặc trưng nhận dạng người. Sự kết hợp giữa YOLO và OSNet giúp hệ thống không chỉ phát hiện được vị trí người trong khung hình mà còn có khả năng phân biệt người mục tiêu với các người khác xuất hiện trong môi trường^[6].

Quy trình tổng quát của thuật toán AI Tracking gồm các bước: khởi tạo mô hình, thu nhận ảnh từ camera, phát hiện người bằng YOLO, khóa mục tiêu ban đầu, tái nhận dạng mục tiêu bằng OSNet, tính toán thông số bám theo và xử lý trường hợp mất mục tiêu.



Hình 3.5 Lưu đồ tổng quát thuật toán AI Tracking

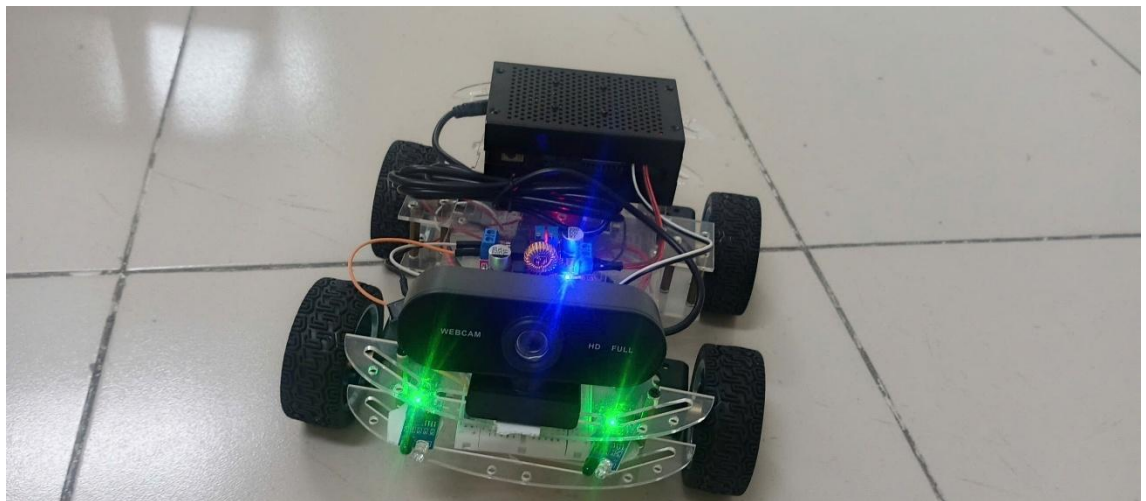
Chương 4

KẾT QUẢ

4.1 KẾT QUẢ MÔ HÌNH THI CÔNG

Sau quá trình thiết kế phần cứng, lắp ráp linh kiện và tích hợp các khối chức năng, mô hình xe tự hành đã được thi công hoàn chỉnh. Hệ thống bao gồm khung xe di động bốn bánh, camera thu nhận hình ảnh, Raspberry Pi làm bộ xử lý trung tâm, mạch nguồn, mạch điều khiển động cơ và các module hỗ trợ truyền tín hiệu điều khiển.

Mô hình xe sau khi hoàn thiện có kết cấu gọn, các thành phần chính được bố trí trên thân xe nhằm đảm bảo khả năng di chuyển ổn định trong môi trường thử nghiệm trong nhà. Camera được lắp ở phía trước xe, đóng vai trò là cảm biến thị giác để thu nhận hình ảnh của mục tiêu. Raspberry Pi được đặt trên thân xe và đảm nhiệm việc xử lý hình ảnh, chạy mô hình AI, cung cấp luồng video cho ứng dụng điều khiển và truyền lệnh điều khiển xuống phần chấp hành.



Hình 4.1 Mô hình xe tự hành sau khi thi công hoàn thiện

Hình 4.1 thể hiện mô hình xe sau khi hoàn thiện phần cứng. Camera được bố trí ở phía trước nhằm quan sát trực tiếp khu vực phía trước xe. Cách bố trí này giúp hệ thống thu nhận hình ảnh của người mục tiêu trong vùng quan sát, từ đó phục vụ

cho quá trình phát hiện, khóa mục tiêu và bám theo mục tiêu. Phần thân xe sử dụng khung mica, giúp cố định các module điện tử và tạo không gian để bố trí dây nối, mạch nguồn, bộ xử lý trung tâm và các thành phần điều khiển.

Về mặt thi công, mô hình đã đạt được các yêu cầu cơ bản sau:

Bảng 4.1 Kết quả thi công các khối chức năng của mô hình

STT	Hạng mục thi công	Kết quả đạt được
1	Khung xe và cơ cấu di chuyển	Đã lắp ráp hoàn chỉnh, xe có thể di chuyển tiến, lùi, rẽ trái, rẽ phải và dừng
2	Camera	Đã lắp đặt ở phía trước xe, thu nhận hình ảnh phục vụ xử lý AI
3	Raspberry Pi	Đã tích hợp vào mô hình, thực hiện vai trò xử lý trung tâm
4	Khối nguồn	Đã cấp nguồn cho các module chính của hệ thống
5	Giao tiếp điều khiển	Đã truyền được lệnh từ ứng dụng đến Raspberry Pi và từ Raspberry Pi xuống khối điều khiển
6	Ứng dụng điều khiển	Đã xây dựng giao diện điều khiển trên máy tính và điện thoại

Từ kết quả thi công, có thể thấy mô hình xe đã được hoàn thiện ở mức prototype, đáp ứng được yêu cầu kiểm thử các chức năng chính của đề tài. Hệ thống có thể nhận hình ảnh từ camera, hiển thị lên ứng dụng điều khiển, chuyển đổi giữa chế độ thủ công và tự động, đồng thời thực hiện các lệnh điều khiển tương ứng theo từng chế độ.

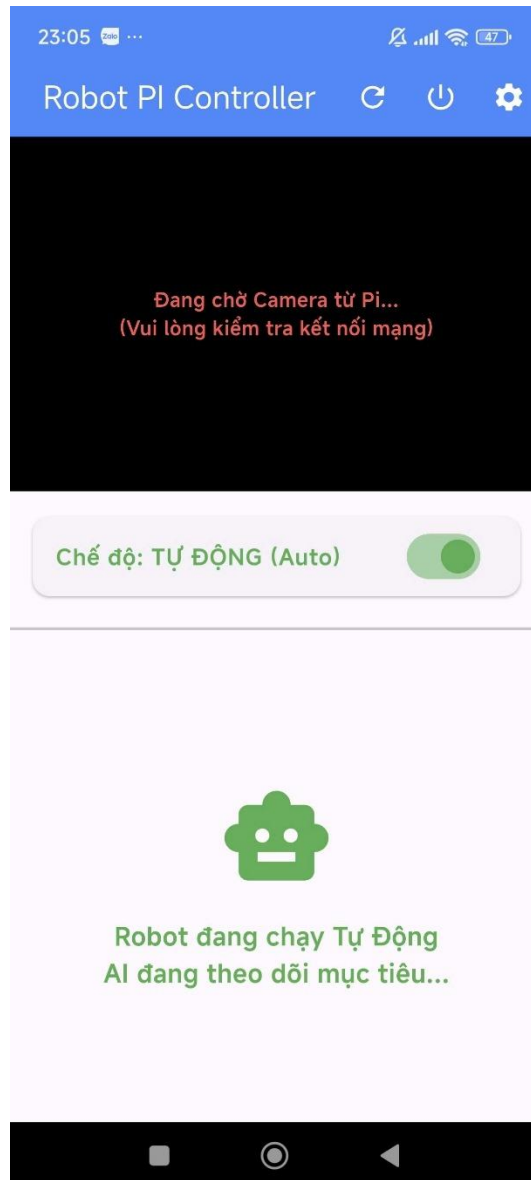
4.2 HOẠT ĐỘNG CỦA HỆ THỐNG

Sau khi hoàn thiện phần cứng và phần mềm, hệ thống được kiểm thử theo các chức năng chính gồm: hiển thị luồng camera, điều khiển thủ công, chuyển đổi chế độ tự động, cấu hình thông số AI và nhận diện mục tiêu bằng mô hình học sâu. Các thử nghiệm được thực hiện trong môi trường trong nhà, trên nền sàn phẳng và có ánh sáng tương đối ổn định.

4.2.1 Hoạt động hiển thị luồng camera

Khi hệ thống được khởi động, Raspberry Pi mở camera và cung cấp luồng video cho ứng dụng điều khiển. Ứng dụng Flutter nhận luồng hình ảnh từ Raspberry Pi và hiển thị trực tiếp trên giao diện người dùng. Chức năng này giúp người vận hành quan sát được vùng nhìn phía trước xe trước khi lựa chọn chế độ điều khiển.

Trong trường hợp kết nối camera hoặc mạng chưa sẵn sàng, ứng dụng hiển thị thông báo chờ camera từ Raspberry Pi. Điều này giúp người dùng nhận biết trạng thái kết nối và kiểm tra lại địa chỉ IP, mạng Wi-Fi hoặc trạng thái server trên Raspberry Pi.

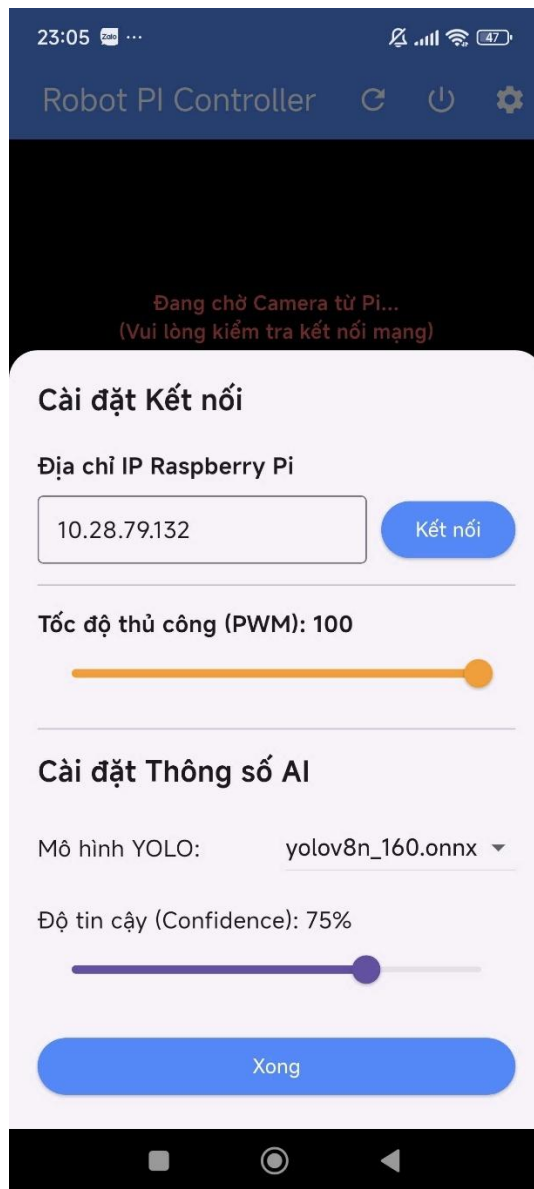


Hình 4.2 Giao diện chờ kết nối camera từ Raspberry Pi

Hình 4.2 minh họa trạng thái ứng dụng khi chưa nhận được luồng camera từ Raspberry Pi. Giao diện hiển thị thông báo “Đang chờ Camera từ Pi” nhằm cảnh báo người dùng kiểm tra lại kết nối mạng. Đây là cơ chế phản hồi cần thiết để tránh trường hợp người dùng thao tác điều khiển khi hệ thống hình ảnh chưa sẵn sàng.

4.2.2 Hoạt động cài đặt kết nối và thông số AI

Ứng dụng điều khiển được thiết kế có phần cài đặt để người dùng nhập địa chỉ IP của Raspberry Pi, điều chỉnh tốc độ thủ công và cấu hình các tham số AI. Các thông số này bao gồm mô hình YOLO được sử dụng và ngưỡng độ tin cậy khi phát hiện đối tượng.

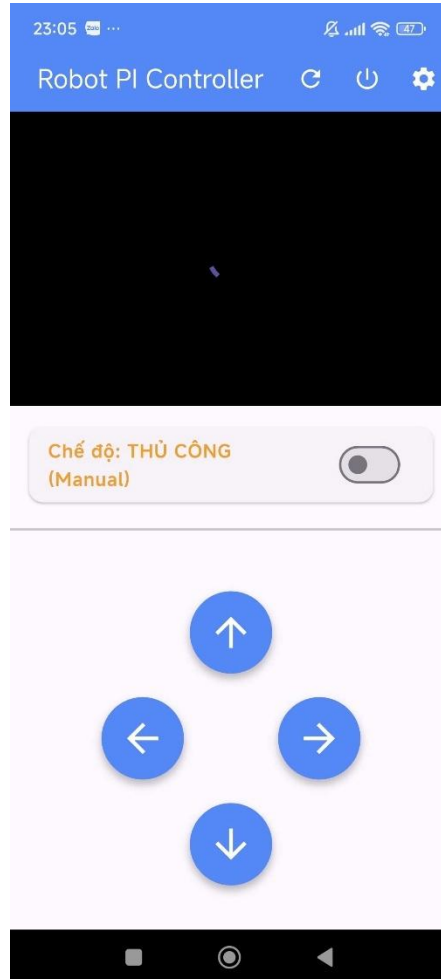


Hình 4.3 Giao diện cài đặt kết nối và thông số AI

Hình 4.3 thể hiện giao diện cài đặt của ứng dụng trên điện thoại. Người dùng có thể nhập địa chỉ IP của Raspberry Pi để thiết lập kết nối đến server điều khiển. Ngoài ra, ứng dụng cho phép điều chỉnh tốc độ thủ công thông qua giá trị PWM, chọn mô hình YOLO và thiết lập ngưỡng confidence. Trong thử nghiệm minh họa, mô hình YOLO được chọn là yolov8n_160.onnx và ngưỡng confidence là 75%. Việc cho phép thay đổi các thông số này giúp hệ thống linh hoạt hơn khi cần cân bằng giữa tốc độ xử lý và độ chính xác phát hiện mục tiêu.

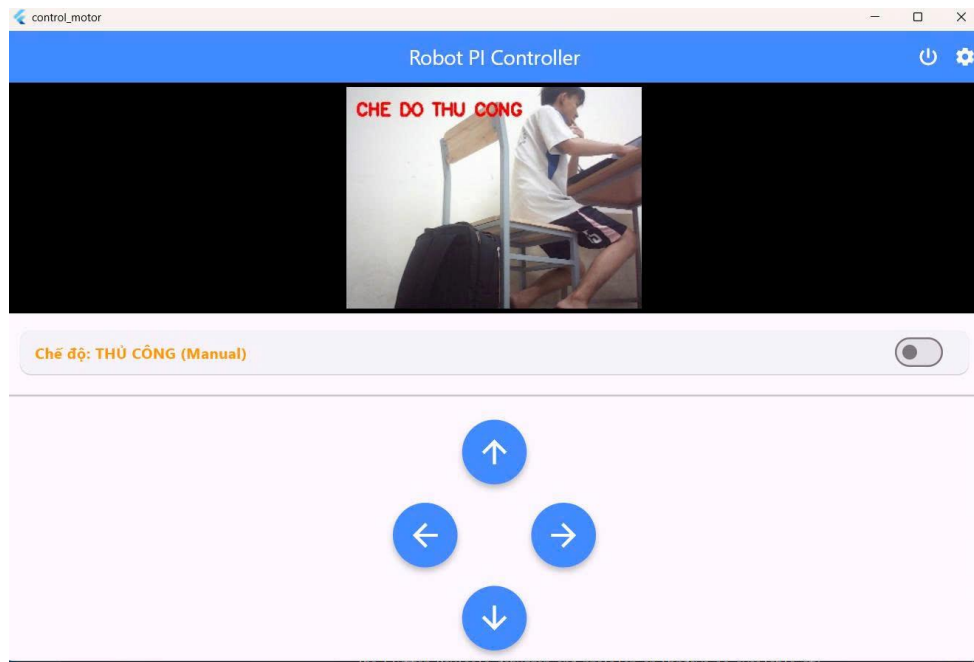
4.2.3 Hoạt động ở chế độ thủ công

Ở chế độ thủ công, người dùng trực tiếp điều khiển xe thông qua các nút điều hướng trên giao diện ứng dụng. Khi người dùng nhấn các nút tiến, lùi, trái hoặc phải, ứng dụng gửi lệnh điều khiển đến Raspberry Pi. Raspberry Pi tiếp nhận lệnh và truyền xuống vi điều khiển để điều khiển động cơ tương ứng.



Hình 4.4 Giao diện điều khiển thủ công trên điện thoại

Hình 4.4 thể hiện giao diện điều khiển thủ công trên điện thoại. Giao diện gồm bốn nút điều hướng tương ứng với các lệnh tiến, lùi, rẽ trái và rẽ phải. Khi công tắc chế độ ở trạng thái Manual, hệ thống cho phép người dùng điều khiển xe trực tiếp. Chế độ này phù hợp cho quá trình kiểm tra phần cứng, kiểm tra động cơ, điều chỉnh vị trí xe hoặc can thiệp khi không muốn sử dụng chế độ tự động.

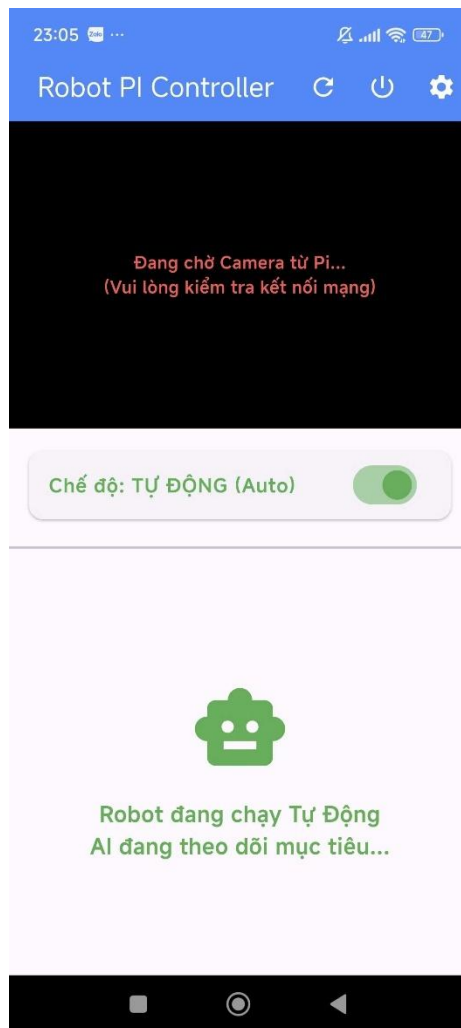


Hình 4.5 Giao diện điều khiển thủ công trên máy tính

Hình 4.5 minh họa giao diện điều khiển thủ công khi ứng dụng chạy trên máy tính. Luồng camera từ Raspberry Pi vẫn được hiển thị ở phía trên, trong khi khu vực điều khiển thủ công được bố trí ở phía dưới. Điều này cho phép người dùng vừa quan sát hình ảnh camera, vừa điều khiển xe theo hướng mong muốn. Kết quả thử nghiệm cho thấy ứng dụng có thể chuyển sang chế độ thủ công và hiển thị đúng trạng thái “THỦ CÔNG (Manual)”.

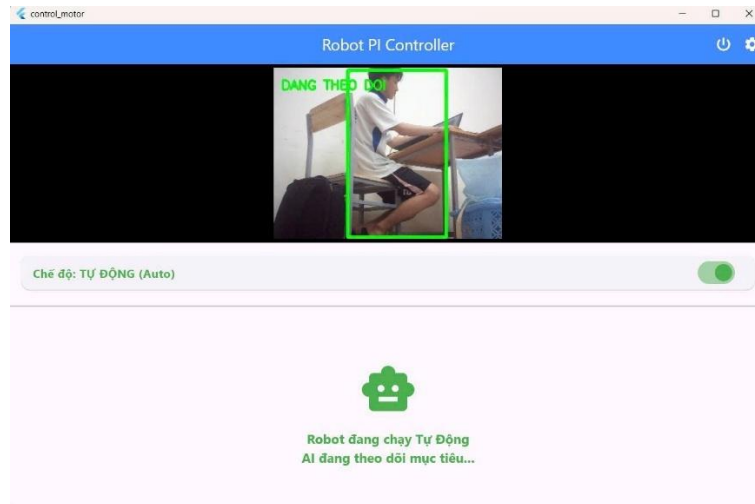
4.2.4 Hoạt động ở chế độ tự động

Ở chế độ tự động, người dùng không điều khiển xe trực tiếp bằng nút. Thay vào đó, Raspberry Pi sử dụng camera để thu nhận hình ảnh, sau đó chạy mô hình AI để phát hiện và theo dõi mục tiêu. Khi phát hiện mục tiêu, hệ thống vẽ bounding box lên đối tượng và hiển thị trạng thái đang theo dõi trên luồng camera.



Hình 4.6 Giao diện chế độ tự động trên điện thoại

Hình 4.7 thể hiện trạng thái ứng dụng khi hệ thống được chuyển sang chế độ tự động. Công tắc chế độ được bật và giao diện hiển thị thông báo “Robot đang chạy Tự Động – AI đang theo dõi mục tiêu”. Ở chế độ này, các nút điều khiển thủ công không được sử dụng, nhằm tránh xung đột giữa lệnh điều khiển từ người dùng và lệnh điều khiển tự động do thuật toán AI sinh ra.



Hình 4.7 Kết quả phát hiện và theo dõi mục tiêu ở chế độ tự động

Hình 4.7 minh họa kết quả hoạt động của hệ thống ở chế độ tự động trên máy tính. Luồng camera hiển thị người mục tiêu trong khung hình, đồng thời hệ thống vẽ bounding box màu xanh lên đối tượng đang được theo dõi. Dòng chữ trạng thái “ĐANG THEO DÕI” cho biết thuật toán AI đã phát hiện được người mục tiêu và đang duy trì quá trình bám theo. Kết quả này cho thấy hệ thống đã thực hiện được chuỗi xử lý chính gồm thu nhận ảnh, phát hiện người, xác định mục tiêu và phản hồi trạng thái theo thời gian thực.

Trong quá trình thử nghiệm, hệ thống tự động hoạt động theo trình tự như sau:

1. Camera thu nhận hình ảnh phía trước xe.
2. Raspberry Pi xử lý từng khung hình từ camera.
3. Mô hình YOLO phát hiện đối tượng người trong ảnh.
4. Hệ thống xác định người mục tiêu và vẽ bounding box.
5. Raspberry Pi tính toán thông tin vị trí của mục tiêu.
6. Lệnh điều khiển được truyền xuống khối điều khiển động cơ.
7. Xe di chuyển để giữ mục tiêu trong vùng quan sát của camera.

Chương 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 KẾT LUẬN

5.1.1 Kết quả đạt được so với mục tiêu đề tài

Sau quá trình nghiên cứu, thiết kế, thi công và kiểm thử, đề tài **“Hệ thống xe tự hành bám sát mục tiêu sử dụng Raspberry Pi”** đã hoàn thành được các mục tiêu chính đặt ra ban đầu. Hệ thống đã xây dựng được một mô hình xe tự hành ở mức prototype, có khả năng thu nhận hình ảnh từ camera, xử lý hình ảnh bằng Raspberry Pi, phát hiện người bằng mô hình YOLO, hỗ trợ tái nhận dạng mục tiêu bằng OSNet và truyền lệnh điều khiển xuống khối chấp hành để điều khiển chuyển động của xe.

Về phần cứng, đề tài đã thi công được mô hình xe di động gồm khung xe, camera, Raspberry Pi, vi điều khiển, mạch điều khiển động cơ, khối nguồn và các động cơ DC. Camera được bố trí ở phía trước xe để thu nhận hình ảnh môi trường. Raspberry Pi đảm nhiệm vai trò xử lý trung tâm, thực hiện các tác vụ như nhận hình ảnh, chạy server, xử lý AI, truyền video về ứng dụng điều khiển và gửi lệnh điều khiển xuống vi điều khiển. Vi điều khiển tiếp nhận lệnh từ Raspberry Pi thông qua UART và điều khiển động cơ để xe thực hiện các trạng thái di chuyển cơ bản như tiến, lùi, rẽ trái, rẽ phải và dừng.

Về phần mềm, hệ thống đã xây dựng được ứng dụng điều khiển bằng Flutter, có thể chạy trên điện thoại và máy tính. Ứng dụng cho phép người dùng quan sát luồng camera từ Raspberry Pi, cấu hình địa chỉ IP kết nối, chuyển đổi giữa chế độ thủ công và chế độ tự động, điều chỉnh tốc độ điều khiển thủ công, chọn mô hình YOLO và thiết lập ngưỡng confidence cho quá trình phát hiện mục tiêu. Việc xây

dụng ứng dụng điều khiển đa nền tảng giúp quá trình vận hành, kiểm thử và trình diễn hệ thống thuận tiện hơn.

Ở chế độ thủ công, người dùng có thể điều khiển xe trực tiếp thông qua các nút điều hướng trên giao diện ứng dụng. Lệnh điều khiển từ ứng dụng được gửi đến Raspberry Pi thông qua Socket.IO, sau đó Raspberry Pi truyền lệnh xuống vi điều khiển qua UART. Kết quả kiểm thử cho thấy hệ thống có thể nhận và thực thi các lệnh điều khiển cơ bản, đáp ứng yêu cầu vận hành xe trong trường hợp người dùng cần kiểm soát trực tiếp.

Ở chế độ tự động, hệ thống sử dụng camera để thu nhận khung hình, sau đó Raspberry Pi xử lý ảnh bằng mô hình YOLO để phát hiện người trong khung hình. Khi phát hiện được mục tiêu, hệ thống hiển thị bounding box lên luồng camera và thể hiện trạng thái đang theo dõi trên giao diện ứng dụng. Ngoài ra, hệ thống còn tích hợp mô hình OSNet để trích xuất đặc trưng ngoại hình, phục vụ cho bài toán tái nhận dạng người mục tiêu trong quá trình bám theo.

Các kết quả trên cho thấy đề tài đã bám sát mục tiêu ban đầu là thiết kế và thi công một hệ thống xe tự hành có khả năng nhận diện, theo dõi và bám theo người mục tiêu trong thời gian thực, sử dụng Raspberry Pi làm bộ xử lý trung tâm, kết hợp YOLO26 và OSNet . Đồng thời, hệ thống cũng đáp ứng được các mục tiêu cụ thể như tích hợp camera, triển khai YOLO, xây dựng pipeline tái nhận dạng bằng OSNet, thiết kế thuật toán điều khiển dựa trên sai lệch vị trí mục tiêu và đánh giá hoạt động của hệ thống thông qua thực nghiệm .

5.1.2 Ưu điểm của hệ thống

Hệ thống có ưu điểm đầu tiên là kiến trúc được phân chia tương đối rõ ràng giữa các khối chức năng. Ứng dụng Flutter đảm nhiệm vai trò giao diện điều khiển và giám sát. Raspberry Pi đảm nhiệm vai trò xử lý trung tâm, chạy server, xử lý camera và thực thi thuật toán AI. Vi điều khiển đảm nhiệm vai trò nhận lệnh điều khiển và điều khiển động cơ. Cách phân chia này giúp hệ thống dễ kiểm thử, dễ sửa lỗi và có khả năng mở rộng trong tương lai.

Hệ thống hỗ trợ hai chế độ hoạt động: thủ công và tự động. Ở chế độ thủ công, người dùng có thể trực tiếp điều khiển hướng di chuyển của xe. Chế độ này hữu

ích trong quá trình kiểm tra phần cứng, đưa xe về vị trí mong muốn hoặc can thiệp khi chế độ tự động chưa phù hợp. Ở chế độ tự động, hệ thống có thể sử dụng camera và thuật toán AI để phát hiện, theo dõi mục tiêu, từ đó tạo nền tảng cho bài toán xe tự hành bám theo người.

Ứng dụng điều khiển được xây dựng bằng Flutter nên có khả năng chạy trên nhiều nền tảng. Trong quá trình thử nghiệm, giao diện có thể hoạt động trên điện thoại và máy tính, giúp người dùng linh hoạt hơn khi vận hành hệ thống. Giao diện ứng dụng cũng được thiết kế trực quan, gồm vùng hiển thị camera, công tắc chọn chế độ, khu vực điều khiển thủ công, trạng thái tự động và bảng cài đặt thông số AI.

Hệ thống sử dụng Socket.IO để truyền lệnh điều khiển giữa ứng dụng và Raspberry Pi. Cơ chế này phù hợp với các yêu cầu điều khiển thời gian thực ở mức mô hình thử nghiệm, cho phép ứng dụng gửi lệnh manual, gửi cấu hình chế độ hoạt động, thay đổi model AI và cập nhật ngưỡng confidence. Bên cạnh đó, video stream từ Raspberry Pi được hiển thị trực tiếp trên ứng dụng, giúp người dùng quan sát được môi trường phía trước xe trong quá trình vận hành.

Hệ thống đã tích hợp được các mô hình AI nhẹ, phù hợp với nền tảng nhúng. YOLO được sử dụng để phát hiện người trong khung hình, trong khi OSNet được sử dụng để hỗ trợ tái nhận dạng người mục tiêu. Việc kết hợp hai mô hình này giúp hệ thống không chỉ phát hiện được đối tượng người mà còn có cơ sở để phân biệt mục tiêu cần theo dõi với các đối tượng khác trong môi trường.

5.1.3 Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế nhất định do giới hạn về thời gian thực hiện, chi phí phần cứng và năng lực tính toán của thiết bị nhúng.

Hiệu năng xử lý AI trên Raspberry Pi còn giới hạn. Raspberry Pi có thể chạy camera, server và mô hình AI, tuy nhiên khi thực hiện đồng thời nhiều tác vụ, tốc độ xử lý có thể bị ảnh hưởng. Điều này làm cho luồng hình ảnh hoặc phản hồi điều khiển chưa thật sự mượt trong một số tình huống. Đặc biệt, khi sử dụng mô hình

có kích thước input lớn hoặc khi môi trường có nhiều đối tượng xuất hiện, thời gian xử lý mỗi khung hình có thể tăng lên.

Chất lượng nhận diện mục tiêu phụ thuộc nhiều vào điều kiện môi trường. Hệ thống hoạt động tốt hơn trong môi trường trong nhà, ánh sáng tương đối ổn định và nền sàn phẳng. Khi ánh sáng yếu, ánh sáng thay đổi mạnh, camera bị rung hoặc mục tiêu bị che khuất, khả năng phát hiện và theo dõi có thể giảm. Đây cũng là giới hạn đã được xác định trong phạm vi đề tài, khi hệ thống chủ yếu hướng đến môi trường indoor và chưa xét đến các điều kiện ngoài trời phức tạp .

Hệ thống hiện mới tập trung vào bài toán bám theo người, chưa tích hợp chức năng tránh vật cản nâng cao. Khi xe di chuyển, nếu có vật cản xuất hiện phía trước, hệ thống chưa có đầy đủ cảm biến chuyên dụng như LiDAR, camera chiều sâu hoặc cảm biến siêu âm để tự động lập kế hoạch tránh vật cản. Vì vậy, trong quá trình thử nghiệm, xe vẫn cần được vận hành trong không gian tương đối trống và có sự giám sát của người dùng.

Thuật toán điều khiển động cơ hiện tại còn ở mức cơ bản. Xe đã có thể di chuyển theo các lệnh tiến, lùi, rẽ trái, rẽ phải và dừng, tuy nhiên chưa có vòng điều khiển kín sử dụng encoder để đo tốc độ thực tế của bánh xe. Do đó, khi mặt sàn thay đổi ma sát, pin yếu hoặc tải trọng thay đổi, vận tốc thực tế của hai bên bánh xe có thể không hoàn toàn đồng đều, dẫn đến sai lệch hướng di chuyển.

Mô hình hiện tại vẫn ở mức prototype phục vụ nghiên cứu và kiểm thử. Phần cơ khí, bố trí dây dẫn, bảo vệ nguồn, độ bền module và khả năng hoạt động liên tục chưa được tối ưu như một sản phẩm hoàn chỉnh. Hệ thống đã chứng minh được nguyên lý hoạt động, nhưng vẫn cần cải tiến thêm nếu muốn ứng dụng trong môi trường thực tế có yêu cầu cao hơn về độ ổn định, an toàn và thời gian vận hành.

5.1.4 Kết luận chung

Từ các kết quả đã đạt được, có thể kết luận rằng đề tài đã hoàn thành được mục tiêu cốt lõi là xây dựng một mô hình xe tự hành bám sát mục tiêu sử dụng Raspberry Pi. Hệ thống đã kết hợp được nhiều thành phần quan trọng gồm phần cứng robot di động, camera, xử lý ảnh, mô hình học sâu, giao tiếp mạng, ứng dụng điều khiển và điều khiển nhúng. Đây là một hệ thống tổng hợp, thể hiện sự kết nối

giữa kiến thức về hệ thống nhúng, trí tuệ nhân tạo, điều khiển tự động và phát triển phần mềm ứng dụng.

Đề tài đã chứng minh được khả năng sử dụng Raspberry Pi làm bộ xử lý trung tâm cho một mô hình xe bám theo người trong môi trường trong nhà. Hệ thống có thể thu nhận hình ảnh từ camera, hiển thị video lên ứng dụng điều khiển, phát hiện người trong khung hình, chuyển đổi giữa chế độ thủ công và tự động, đồng thời truyền lệnh điều khiển xuống khối chấp hành. Các kết quả thực nghiệm ở Chương 4 cho thấy mô hình đã hoạt động đúng theo các chức năng chính được thiết kế.

Mặc dù vẫn còn một số hạn chế về tốc độ xử lý, độ ổn định trong môi trường phức tạp, khả năng tránh vật cản và độ hoàn thiện cơ khí, đề tài đã tạo được nền tảng ban đầu cho việc phát triển các hệ thống robot hỗ trợ con người trong môi trường trong nhà. Những kết quả này là cơ sở để tiếp tục cải tiến hệ thống theo hướng tối ưu hiệu năng AI, nâng cao độ ổn định điều khiển, bổ sung cảm biến an toàn và mở rộng khả năng ứng dụng thực tế trong tương lai.

5.2 HƯỚNG PHÁT TRIỂN

Từ các kết quả đã đạt được và những hạn chế còn tồn tại trong quá trình thực nghiệm, hệ thống có thể tiếp tục được phát triển theo nhiều hướng nhằm nâng cao hiệu năng xử lý, độ ổn định khi bám theo mục tiêu, khả năng vận hành an toàn và mức độ hoàn thiện của mô hình. Các hướng phát triển dưới đây tập trung trực tiếp vào các thành phần chính của đề tài, bao gồm mô hình AI trên Raspberry Pi, thuật toán bám theo người, khả năng tránh vật cản, hệ thống điều khiển động cơ, nguồn cấp và ứng dụng điều khiển.

5.2.1 Tối ưu hiệu năng xử lý AI

Trong hệ thống hiện tại, Raspberry Pi đảm nhiệm đồng thời nhiều tác vụ như thu nhận hình ảnh từ camera, cung cấp luồng video MJPEG cho ứng dụng điều khiển, nhận lệnh Socket.IO, chạy mô hình YOLO để phát hiện người và chạy mô hình OSNet để trích xuất đặc trưng phục vụ tái nhận dạng mục tiêu. Việc xử lý nhiều tác vụ cùng lúc trên một thiết bị nhúng có tài nguyên tính toán giới hạn làm cho tốc độ xử lý AI chưa thật sự tối ưu, đặc biệt khi sử dụng mô hình có kích thước đầu vào lớn hoặc khi trong khung hình xuất hiện nhiều người.

Hướng phát triển đầu tiên là tiếp tục tối ưu mô hình YOLO đang sử dụng. Trong phiên bản hiện tại, hệ thống đã hỗ trợ lựa chọn các mô hình YOLO theo nhiều kích thước như `yolo26n_160.onnx`, `yolo26n_320.onnx` và `yolo26n_640.onnx`. Tuy nhiên, cần thực hiện đánh giá định lượng giữa các phiên bản này để chọn cấu hình phù hợp nhất cho Raspberry Pi. Cụ thể, cần đo tốc độ xử lý trung bình, độ trễ mỗi khung hình và khả năng phát hiện mục tiêu ở từng kích thước đầu vào. Nếu ưu tiên tốc độ, mô hình kích thước nhỏ như 160x160 có thể được sử dụng trong điều kiện mục tiêu ở gần và môi trường ít nhiễu. Nếu cần độ chính xác cao hơn, có thể sử dụng mô hình 320x320 nhưng cần cân nhắc độ trễ xử lý.

Bên cạnh đó, có thể áp dụng các kỹ thuật tối ưu mô hình như lượng tử hóa trọng số từ FP32 sang FP16 hoặc INT8, chuyển đổi mô hình sang định dạng tối ưu cho thiết bị biên, hoặc sử dụng các runtime có khả năng tăng tốc tốt hơn trên ARM. Việc lượng tử hóa có thể giúp giảm dung lượng mô hình, giảm thời gian suy luận và giảm tải cho CPU của Raspberry Pi. Tuy nhiên, sau khi tối ưu cần kiểm tra lại độ chính xác phát hiện để tránh trường hợp mô hình chạy nhanh hơn nhưng bỏ sót mục tiêu hoặc phát hiện sai nhiều hơn.

Đối với mô hình OSNet, việc trích xuất đặc trưng cho từng người trong khung hình cũng tiêu tốn thời gian xử lý. Do đó, hướng cải tiến phù hợp là không chạy Re-ID ở mọi khung hình, mà chỉ thực hiện tái nhận dạng theo chu kỳ hoặc khi có nhiều người xuất hiện đồng thời. Trong trường hợp chỉ có một người trong khung hình và mục tiêu đang được theo dõi ổn định, hệ thống có thể ưu tiên dùng vị trí bounding box liên tiếp để duy trì tracking, sau đó chỉ kích hoạt OSNet khi mục tiêu bị mất, bị che khuất hoặc có người khác đi vào vùng quan sát. Cách xử lý này giúp giảm số lần chạy Re-ID, từ đó cải thiện tốc độ xử lý tổng thể.

Ngoài tối ưu mô hình, hệ thống cũng cần tối ưu pipeline xử lý ảnh. Camera hiện được cấu hình ở độ phân giải thấp để phù hợp với Raspberry Pi. Trong tương lai, có thể tách riêng luồng xử lý AI và luồng hiển thị camera. Luồng AI có thể sử dụng ảnh resize kích thước nhỏ để suy luận nhanh, trong khi luồng hiển thị cho

ứng dụng vẫn giữ chất lượng đủ quan sát. Việc tách hai luồng này giúp tránh tình trạng video stream và AI inference ảnh hưởng lẫn nhau.

Một hướng phát triển khác là bổ sung cơ chế đo FPS, thời gian suy luận và độ trễ truyền hình ảnh ngay trên ứng dụng điều khiển. Khi có các thông số này, người vận hành có thể đánh giá trực tiếp hệ thống đang chạy ở cấu hình nào là ổn định nhất. Đây cũng là cơ sở để so sánh hiệu quả trước và sau khi tối ưu mô hình AI.

5.2.2 Cải thiện thuật toán bám theo mục tiêu

Thuật toán bám theo mục tiêu hiện tại dựa trên thông tin bounding box của người được phát hiện trong khung hình. Hệ thống tính toán vị trí tâm của bounding box so với tâm ảnh để xác định sai lệch ngang, đồng thời dùng chiều cao hoặc diện tích bounding box để ước lượng tương đối khoảng cách giữa xe và mục tiêu. Cách tiếp cận này phù hợp với mô hình prototype, tuy nhiên vẫn còn hạn chế khi mục tiêu di chuyển nhanh, bị che khuất tạm thời hoặc khi có nhiều người xuất hiện trong cùng khung hình.

Hướng phát triển quan trọng là bổ sung cơ chế theo dõi liên tục qua nhiều khung hình thay vì chỉ xử lý từng frame độc lập. Có thể sử dụng bộ lọc Kalman để dự đoán vị trí tiếp theo của mục tiêu dựa trên vị trí và vận tốc thay đổi trong các khung hình trước đó. Khi mục tiêu bị che khuất trong thời gian ngắn hoặc mô hình YOLO không phát hiện được ở một vài frame, bộ lọc dự đoán vẫn có thể cung cấp vị trí ước lượng để xe không dừng đột ngột. Điều này giúp quá trình bám theo mượt hơn và giảm hiện tượng mất mục tiêu tạm thời.

Ngoài ra, hệ thống có thể tích hợp thêm các thuật toán tracking như SORT, DeepSORT hoặc ByteTrack^[7]. Trong đó, YOLO đảm nhiệm nhiệm vụ phát hiện người, còn thuật toán tracking đảm nhiệm việc gán ID và duy trì quỹ đạo của từng người qua thời gian. Khi kết hợp với OSNet, hệ thống có thể vừa theo dõi được vị trí mục tiêu, vừa kiểm tra lại đặc trưng ngoại hình khi có nguy cơ nhầm lẫn giữa nhiều người. Cách kết hợp này phù hợp với bài toán bám theo người trong môi trường có nhiều đối tượng di chuyển.

Một hướng cải tiến khác là hoàn thiện cơ chế khóa mục tiêu ban đầu. Hiện tại, hệ thống có thể chọn mục tiêu dựa trên người có bounding box lớn nhất hoặc người

xuất hiện rõ nhất trong khung hình. Tuy nhiên, trong thực tế, người dùng cần có cách chủ động chọn mục tiêu. Do đó, ứng dụng điều khiển có thể bổ sung chức năng “khóa mục tiêu” bằng cách chạm vào người cần theo dõi trên màn hình camera. Khi người dùng chọn vùng mục tiêu, Raspberry Pi sẽ lưu đặc trưng ngoại hình của người đó bằng OSNet và sử dụng đặc trưng này làm mẫu đối chiếu trong quá trình bám theo.

Thuật toán điều khiển bám theo cũng cần được cải thiện để xe di chuyển ổn định hơn. Thay vì chỉ chia vùng điều khiển đơn giản như lệch trái, lệch phải, đi thẳng hoặc dừng, hệ thống có thể dùng luật điều khiển tỉ lệ dựa trên độ lệch $offset_x$. Khi mục tiêu lệch càng xa tâm ảnh, tốc độ chênh lệch giữa hai bánh càng lớn; khi mục tiêu gần tâm ảnh, xe giảm mức điều chỉnh để tránh rung lắc. Tương tự, khoảng cách đến mục tiêu có thể được điều khiển dựa trên tỉ lệ chiều cao bounding box. Nếu mục tiêu quá xa, xe tăng tốc tiến; nếu mục tiêu quá gần, xe giảm tốc hoặc dừng lại. Cách điều khiển liên tục này sẽ giúp xe bám theo tự nhiên hơn so với điều khiển theo ngưỡng rời rạc.

Bên cạnh đó, cần bổ sung trạng thái xử lý khi mất mục tiêu. Thay vì dừng ngay lập tức sau một frame không phát hiện được người, hệ thống có thể duy trì trạng thái “mất mục tiêu tạm thời” trong một khoảng thời gian ngắn. Trong thời gian này, xe có thể giảm tốc, giữ hướng cũ hoặc quay nhẹ để tìm lại mục tiêu. Nếu sau thời gian chờ vẫn không tìm thấy mục tiêu, xe mới chuyển sang trạng thái dừng an toàn. Điều này giúp hệ thống bớt nhạy với lỗi phát hiện tức thời của mô hình AI.

5.2.3 Bổ sung chức năng tránh vật cản

Trong phạm vi hiện tại, hệ thống tập trung vào bài toán nhận diện và bám theo người mục tiêu bằng camera. Tuy nhiên, khi xe di chuyển trong môi trường thực tế, phía trước xe có thể xuất hiện vật cản như tường, bàn ghế, người khác, đồ vật dưới sàn hoặc các chướng ngại bất ngờ. Nếu chỉ dựa vào camera và thuật toán bám mục tiêu, xe có thể tiếp tục di chuyển về phía mục tiêu mà chưa đánh giá đầy đủ nguy cơ va chạm. Vì vậy, bổ sung chức năng tránh vật cản là hướng phát triển cần thiết để tăng tính an toàn cho hệ thống.

Hướng cải tiến đơn giản nhất là tích hợp cảm biến siêu âm hoặc cảm biến hồng ngoại ở phía trước xe. Các cảm biến này có thể đo khoảng cách từ xe đến vật cản trong vùng gần. Khi khoảng cách nhỏ hơn ngưỡng an toàn, vi điều khiển hoặc Raspberry Pi có thể ưu tiên lệnh dừng, bất kể thuật toán AI đang yêu cầu xe tiến về phía mục tiêu. Cơ chế ưu tiên an toàn này giúp giảm nguy cơ va chạm trong các tình huống vật cản xuất hiện đột ngột.

Để tăng độ bao phủ, có thể bố trí nhiều cảm biến ở các hướng khác nhau, ví dụ phía trước trái, phía trước giữa và phía trước phải. Khi cảm biến giữa phát hiện vật cản, xe dừng hoặc giảm tốc. Khi cảm biến bên trái phát hiện vật cản gần hơn, xe có thể hạn chế rẽ trái. Khi cảm biến bên phải phát hiện vật cản, xe hạn chế rẽ phải. Cách bố trí này giúp xe không chỉ dừng mà còn có thể chọn hướng điều chỉnh phù hợp hơn khi bám theo mục tiêu trong không gian hẹp.

Ở mức phát triển cao hơn, có thể kết hợp thông tin từ camera với mô hình phát hiện vật cản. Ngoài lớp “person”, YOLO có thể được mở rộng để nhận diện thêm các đối tượng thường gặp trong môi trường trong nhà như ghế, bàn, balo hoặc vật thể lớn trước xe. Khi phát hiện vật cản nằm trong vùng di chuyển, hệ thống có thể giảm tốc hoặc dừng lại. Tuy nhiên, cách này cần được kết hợp với cảm biến khoảng cách, vì ảnh 2D từ camera thường khó xác định chính xác khoảng cách thật đến vật cản nếu không có camera chiều sâu.

Một hướng phát triển tốt hơn là sử dụng camera chiều sâu hoặc LiDAR. Camera chiều sâu giúp hệ thống biết được khoảng cách từ xe đến từng vùng trong ảnh, từ đó phân biệt mục tiêu cần bám theo và vật cản ở gần xe. LiDAR có thể cung cấp thông tin khoảng cách ổn định hơn, hỗ trợ phát hiện vật cản theo nhiều hướng. Tuy nhiên, các giải pháp này làm tăng chi phí và độ phức tạp của hệ thống, nên phù hợp với các phiên bản phát triển sau khi mô hình prototype đã hoạt động ổn định.

Khi bổ sung tránh vật cản, cần thiết kế lại logic ưu tiên điều khiển. Lệnh điều khiển từ AI Tracking không nên được gửi trực tiếp xuống động cơ nếu hệ thống phát hiện nguy cơ va chạm. Thay vào đó, cần có một tầng kiểm tra an toàn nằm giữa thuật toán bám mục tiêu và khối điều khiển động cơ. Tầng này nhận lệnh

mong muốn từ AI, kiểm tra dữ liệu vật cản, sau đó quyết định cho phép di chuyển, giảm tốc, đổi hướng hoặc dừng khẩn cấp. Đây là yêu cầu quan trọng để hệ thống có thể vận hành an toàn hơn trong môi trường thực tế.

5.2.4 Nâng cấp điều khiển động cơ và hệ thống nguồn

Hệ thống hiện tại đã có thể điều khiển xe thực hiện các chuyển động cơ bản như tiến, lùi, rẽ trái, rẽ phải và dừng. Tuy nhiên, điều khiển động cơ vẫn còn ở mức cơ bản, chủ yếu dựa vào lệnh hướng và tốc độ được gửi từ Raspberry Pi hoặc ứng dụng điều khiển. Trong thực tế, hai động cơ có thể không quay hoàn toàn giống nhau dù nhận cùng giá trị PWM, do sai số cơ khí, độ mòn động cơ, tải trọng, độ ma sát mặt sàn hoặc mức pin thay đổi. Điều này có thể làm xe bị lệch hướng khi đi thẳng hoặc phản ứng không đều khi rẽ.

Hướng phát triển cần thiết là bổ sung encoder cho từng động cơ để đo tốc độ quay thực tế của bánh xe. Khi có dữ liệu phản hồi từ encoder, vi điều khiển có thể xây dựng vòng điều khiển kín cho từng bánh. Bộ điều khiển PID có thể được dùng để điều chỉnh PWM sao cho tốc độ thực tế của động cơ bám sát tốc độ mong muốn. Nhờ đó, xe có thể đi thẳng ổn định hơn, rẽ chính xác hơn và phản ứng tốt hơn khi tải trọng hoặc điều kiện mặt sàn thay đổi.

Đối với chế độ tự động, việc có encoder cũng giúp cải thiện thuật toán bám mục tiêu. Khi Raspberry Pi gửi thông tin sai lệch mục tiêu xuống vi điều khiển, vi điều khiển có thể chuyển đổi thông tin này thành vận tốc mong muốn cho bánh trái và bánh phải. Sau đó, bộ điều khiển PID đảm bảo hai bánh đạt đúng vận tốc yêu cầu. Cách làm này tách rõ phần xử lý AI ở Raspberry Pi và phần điều khiển thời gian thực ở vi điều khiển, giúp hệ thống ổn định hơn so với việc chỉ gửi lệnh điều khiển đơn giản.

Ngoài điều khiển động cơ, hệ thống nguồn cũng cần được nâng cấp. Raspberry Pi là thiết bị nhạy với sụt áp, trong khi động cơ DC thường tạo dòng khởi động lớn và nhiễu điện khi thay đổi tốc độ. Nếu nguồn cấp cho Raspberry Pi và nguồn cấp cho động cơ không được tách và lọc tốt, hệ thống có thể gặp hiện tượng treo, mất kết nối camera hoặc hoạt động không ổn định khi xe tăng tốc. Vì vậy, cần thiết kế lại hệ thống nguồn theo hướng tách riêng nguồn logic và nguồn công suất.

Nguồn cấp cho Raspberry Pi nên sử dụng bộ chuyển đổi DC-DC ổn định, có dòng ra đủ lớn và có tụ lọc gần đầu cấp nguồn. Nguồn cấp cho động cơ nên đi qua mạch công suất riêng, có khả năng chịu dòng khởi động của động cơ. Cần bổ sung tụ lọc dung lượng phù hợp ở đầu vào motor driver để giảm sụt áp tức thời. Ngoài ra, dây nguồn cho động cơ và dây nguồn cho Raspberry Pi nên được bố trí tách biệt để hạn chế nhiều ảnh hưởng đến khối xử lý.

Một hướng cải tiến khác là thay thế driver L298N bằng driver hiệu suất cao hơn. L298N dễ sử dụng và phù hợp với mô hình thử nghiệm, nhưng có nhược điểm là sụt áp lớn và hiệu suất không cao. Trong các phiên bản sau, có thể sử dụng driver dùng MOSFET như TB6612FNG, BTS7960 hoặc các module điều khiển động cơ có dòng chịu tải tốt hơn. Việc thay driver sẽ giúp giảm tổn hao nhiệt, tăng hiệu suất cấp nguồn cho động cơ và cải thiện thời gian hoạt động của pin.

Ngoài ra, cần bổ sung các cơ chế bảo vệ nguồn như công tắc nguồn tổng, cầu chì hoặc mạch bảo vệ quá dòng, mạch đo điện áp pin và cảnh báo pin yếu. Khi điện áp pin xuống thấp, hệ thống có thể cảnh báo trên ứng dụng hoặc tự động dừng xe để tránh tình trạng Raspberry Pi tắt đột ngột. Đây là yếu tố quan trọng nếu muốn hệ thống hoạt động an toàn và ổn định trong thời gian dài.

5.2.5 Hoàn thiện ứng dụng điều khiển và khả năng ứng dụng thực tế

Ứng dụng điều khiển Flutter hiện đã đáp ứng được các chức năng chính gồm hiển thị camera, chuyển đổi chế độ Manual/Auto, điều khiển thủ công, nhập địa chỉ IP Raspberry Pi, điều chỉnh tốc độ PWM, chọn mô hình YOLO và chỉnh ngưỡng confidence. Tuy nhiên, để hệ thống dễ sử dụng và phù hợp hơn với thực nghiệm thực tế, ứng dụng vẫn cần được hoàn thiện thêm về giao diện, trạng thái hệ thống và khả năng giám sát.

Trong phiên bản hiện tại, người dùng nhập địa chỉ IP Raspberry Pi để kết nối. Ở các phiên bản sau, ứng dụng có thể lưu lại IP đã sử dụng gần nhất, tự động kết nối lại khi mở app hoặc hỗ trợ quét thiết bị trong cùng mạng LAN. Nếu Raspberry Pi được cấu hình phát Wi-Fi hotspot, ứng dụng có thể hướng dẫn người dùng kết

nối trực tiếp vào mạng của xe, giúp quá trình vận hành độc lập hơn và không phụ thuộc vào router bên ngoài.

Ứng dụng có thể bổ sung chức năng chọn mục tiêu trực tiếp trên màn hình camera. Đây là cải tiến rất phù hợp với đề tài bám theo người. Khi có nhiều người trong khung hình, người dùng có thể chạm vào người cần theo dõi. Ứng dụng gửi tọa độ vùng chọn xuống Raspberry Pi, sau đó Raspberry Pi dùng vùng ảnh tương ứng để trích xuất đặc trưng OSNet và khóa mục tiêu. Chức năng này giúp quá trình chọn mục tiêu chủ động và chính xác hơn so với việc hệ thống tự chọn người có bounding box lớn nhất.

Cần bổ sung các nút an toàn trên giao diện như dừng khẩn cấp, ngắt điều khiển tự động và tắt Raspberry Pi an toàn. Nút dừng khẩn cấp nên được ưu tiên cao nhất, khi nhấn sẽ gửi lệnh dừng ngay đến Raspberry Pi và vi điều khiển. Chức năng tắt Raspberry Pi an toàn cũng cần được duy trì để tránh việc ngắt nguồn đột ngột gây lỗi hệ điều hành hoặc hỏng dữ liệu trên thẻ nhớ.

Về khả năng ứng dụng thực tế, hệ thống có thể tiếp tục phát triển theo hướng robot hỗ trợ vận chuyển vật dụng trong môi trường trong nhà như phòng học, phòng thí nghiệm, văn phòng hoặc kho nhỏ. Để đạt được điều này, cần cải tiến thêm về tải trọng, độ ổn định cơ khí, thời lượng pin, khả năng tránh vật cản, khả năng bám theo mục tiêu trong môi trường có nhiều người và độ an toàn khi vận hành gần con người. Ở giai đoạn hiện tại, hệ thống đã chứng minh được nguyên lý hoạt động ở mức prototype; các cải tiến trên sẽ giúp mô hình tiến gần hơn đến một hệ thống robot hỗ trợ thực tế.

Tóm lại, hướng phát triển của đề tài cần tập trung vào năm nhóm chính: tối ưu hiệu năng AI trên Raspberry Pi, cải thiện thuật toán bám mục tiêu, bổ sung tránh vật cản, nâng cấp điều khiển động cơ và nguồn cấp, đồng thời hoàn thiện ứng dụng điều khiển. Những cải tiến này không chỉ khắc phục các hạn chế hiện tại mà còn tạo nền tảng để hệ thống có thể vận hành ổn định hơn, an toàn hơn và có khả năng mở rộng cho các ứng dụng robot hỗ trợ con người trong môi trường trong nhà.

TÀI LIỆU THAM KHẢO

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA, USA: MIT Press, 2011.
- [2] A. Eirale, M. Martini, and M. Chiaberge, “Human following and guidance by autonomous mobile robots: A comprehensive review,” *IEEE Access*, vol. 13, pp. 42214–42253, 2025, doi: 10.1109/ACCESS.2025.3548134.
- [3] F. Chaumette and S. Hutchinson, “Visual servo control. I. Basic approaches,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, Dec. 2006, doi: 10.1109/MRA.2006.250573.
- [4] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. 2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, San Diego, CA, USA, 2005, vol. 1, pp. 886–893, doi: 10.1109/CVPR.2005.177.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788, doi: 10.1109/CVPR.2016.91.
- [6] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, “Simple online and realtime tracking,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, Phoenix, AZ, USA, 2016, pp. 3464–3468, doi: 10.1109/ICIP.2016.7533003.
- [7] N. Wojke, A. Bewley, and D. Paulus, “Simple online and realtime tracking with a deep association metric,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, Beijing, China, 2017, pp. 3645–3649, doi: 10.1109/ICIP.2017.8296962.
- [8] L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang, and Q. Tian, “Scalable Person Re-identification: A Benchmark,” in *Proc. IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1116–1124.

[9] M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao, and S. C. H. Hoi, "Deep learning for person re-identification: A survey and outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 2872–2893, Jun. 2022, doi: 10.1109/TPAMI.2021.3054775.

[10] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 815–823, doi: 10.1109/CVPR.2015.7298682.

[11] K. Zhou, Y. Yang, A. Cavallaro, and T. Xiang, "Omni-Scale Feature Learning for Person Re-Identification," in *Proc. IEEE/CVF International Conference on Computer Vision (ICCV)*, Seoul, Korea (South), 2019, pp. 9700–9709

PHỤ LỤC

Phụ lục sẽ là nơi trình bày các vấn đề phụ như: hướng dẫn sử dụng hệ thống (nếu có), source code, hoặc các vấn đề cần giới thiệu thêm.

